
BASAL-1.0: RELIABLE, HIGHLY OPTIMIZED TYPED DECISIONS FOR POLISH

A CALIBRATED 4.5B MODEL AT 12.5 MS PER DECISION — A STEP-BY-STEP RESEARCH REPORT

Remigiusz Kinas, ai5

ABSTRACT

A typed-decision model reads a state (a message, a document, a case file) and answers a schema-constrained question with a probability distribution over the allowed answers, in one forward pass and without generating text. We present basal-1.0, the first independently evaluated typed-decision models for Polish, and document step by step how they were built. The models are trained on synthetic Polish and English decisions whose labels are computed by code, grounded in statutes or checked by independent verifiers. The answer is read from the option letters, averaged over both option orders and calibrated per decision type. basal-1.0-4.5B reaches 0.884 on held-out Polish decisions against 0.780 for the commercial Jev API and 0.779 for the best of eleven open Jev-like systems; on English decisions it is not distinguishable from Jev (0.741 vs 0.736, paired difference $+0.005$ $[-0.020, +0.029]$). With a confidence threshold fixed on calibration data it decides 58.6% of test decisions automatically at 1.2% observed error (target 1%; Jev under the same procedure: 18.1%). A serving engine that packs both option orders over a shared prefix, with CUDA graphs, compilation and token-budget batching, reduces a calibrated decision from 177 ms to 12.5 ms on an H100 and 8.8 ms on a B300; the distilled basal-1.0-1.5B needs 4.7 ms. Every design choice is backed by an ablation, including negative results: trained early exits save only 1.13–1.16 $\times$ (the trained heads reach the required agreement only in the last ten layers) and are offered as a per-request setting. Models and engine are released under Apache 2.0.

Keywords typed decisions · Polish · calibration · selective prediction · distillation · early exit · model compression

1 Problem and goals

Typed decisions. An application sends a *state* and one or more *typed questions*. Each question lists its allowed answers. Three answer types cover most uses: *choice* (one of k options), *score* (a position on ordered levels) and *nou1* (the probability that a yes/no question is answered “yes”). The model returns probabilities, not text. The application then decides what to do with them, for example to act automatically only when the model is confident and to route the rest to a person.

Goals. We wanted a model that (1) is at least as accurate as a strong commercial typed-decision API on Polish decisions, (2) gives probabilities that can be trusted, measured as the share of decisions that can be accepted while keeping the error among accepted decisions below 1% or 5%, (3) gives the same answer when options are reordered, (4) runs locally and fast, and (5) keeps English ability. Throughout, the reference is the commercial typed-decision API Jev, version 1.13.0 [1] (“Jev” below; all Jev numbers in this report come from that version).

Contributions.

1. **The first independently evaluated typed-decision models for Polish.** basal-1.0-4.5B and basal-1.0-1.5B, released under Apache 2.0 with FP8 and NVFP4 checkpoints. The 4.5B model is the most accurate system on held-out Polish decisions (0.884, against 0.780 for Jev and 0.779 for the best of eleven open Jev-like systems),

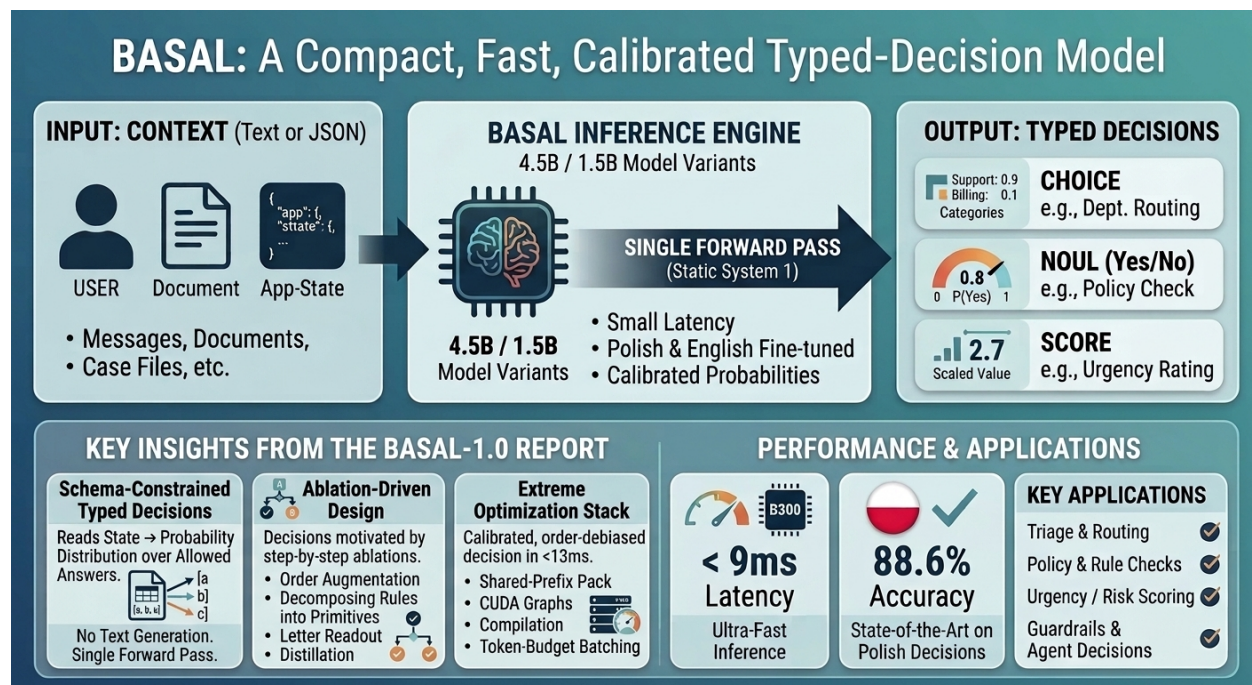


Figure 1: Overview of basal-1.0. A state (text or JSON) and typed questions go in. One forward pass returns calibrated probabilities for choice, noul and score questions. Latency: 8.8 ms per two-order decision on a B300 and 12.5 ms on an H100; accuracy on held-out Polish decisions: 0.884 (Section 20).

is not distinguishable from Jev on English decisions (paired difference $+0.005$ [-0.020 , $+0.029$]) and, with a confidence threshold fixed on calibration data before testing, decides 58.6% of the test decisions automatically at 1.2% observed error (target 1%; Jev under the same procedure: 18.1%).

2. **A data recipe with auditable labels.** Synthetic Polish and English decision data whose labels are computed by code, grounded in statute text or checked by two independent verifier models, with twin pairs (one changed fact changes the answer), evidence-sufficiency pairs and splits by template, statute and domain. Its composition, provenance and a post-release audit of the statutory rules (one wrong rule found and corrected in the evaluation data) are reported group by group (Section 18).
3. **A step-by-step record of design decisions,** each backed by an ablation: tokenizer, letter readout, order augmentation, LoRA versus full fine-tuning, decision distillation, decomposition of composite rules, calibration per decision type, and the negative results that shaped the design (distillation on generic text, a mixture-of-experts base whose router must stay in fp32, layer pruning, 4-bit weights, and a reasoning-data release that improved English at a small cost in Polish).
4. **A serving engine for typed decisions.** bf16, both option orders packed after one shared prefix, CUDA graphs, compilation and token-budget batching take a calibrated, order-averaged decision from 177 ms to 12.5 ms on an H100 (8.8 ms on a B300, 4.7 ms for the 1.5B model on a B300) with unchanged decisions. Trained early exits are offered as a per-request setting. The engine implements the System One JSON interface and its responses validate against the official schema.
5. **Adaptive depth for typed decisions.** To our knowledge (Section 2), the first Jev-like model with trained early exits, and the first reported attempt at a looped (recursive, shared-block) compression of one. A logit-lens analysis on a stratified sample shows that the decision becomes readable through the output head only in the last ten layers (90% agreement at layer 54 of 60), consistent with both results: the exits save only 1.13 – $1.16\times$ and are therefore shipped as a per-request setting ($12.2 \rightarrow 10.5$ ms on an H100 at unchanged agreement), and the looped model cannot recover the replaced layers from decision data alone (Steps 4 and 11).
6. **A measured map of precision and hardware.** bf16 on six GPUs, FP8 on five and NVFP4 on two, from a data-centre B300 to a desktop DGX Spark, showing that the fastest precision depends on whether the machine is limited by kernel launches, compute or memory bandwidth.

7. **A moderation case study from definitions alone.** On five Polish safety benchmarks, basal-1.0-4.5B with one-sentence category definitions and no training on these benchmarks or on Polish moderation data (its fine-tuning data contain 587 English moderation items) is significantly better than Sójka-0.1B on four of five benchmarks and, against the larger Sójka-0.5B, significantly better on two metrics, worse on one and inconclusive on three (paired bootstrap). It also answers questions outside any fixed taxonomy, such as the targeted group (92.5% accuracy) and threats (AUROC 0.91), at 5–10× the single-text latency of the guards (Section 21).
8. **A like-for-like comparison** of Jev and eleven open Jev-like systems on Polish and English decision sets with one pipeline (open systems on one H100), published as a leaderboard that shows scores only, so that the test items cannot be trained on.

How to read this report. Each step states what we did, what we measured, and the decision that followed. Result tables are generated from saved metrics files. The data-composition tables (Section 18), the Step 13 comparison, the JevBench leaderboard excerpt (Table 1), the request examples and the summary of decisions are transcribed from the data manifest, the evaluation outputs and the public leaderboard.

2 Related work: Jev and Jev-like systems

Jev. TypeSafe AI introduced *System One* models and their first member, Jev, in September 2026 [1]. An application sends an unstructured state and a set of typed questions (choice with 2–255 options, an ordered score rubric, or a yes/no/noul) and receives, for each question, a probability distribution over the allowed answers instead of generated text. The answer can never fall outside the schema. TypeSafe describes the training as reinforcement learning for calibrated decisions and reports 70–500 ms per request, but has not published the architecture, parameter count or training data. Its own documentation lists weaknesses in arithmetic, dates and literal reading of questions, and English as the primary language.

An ecosystem within days. Open reimplementations appeared within three days of the release. By 24 September 2026 a community list counted 896 related projects, 60 of them open Jev-like models [2], and JevBench, an independent benchmark with frozen and hashed items, ranked 71 systems [3, 4]. What spread fastest is the interface: at least eight projects implement the same `POST /v1/systemone` contract, with the aim that the official client works after changing the base URL. The open systems follow five designs:

- **Training-free letter readout.** Render the options as lettered choices, prefill the answer prefix and read the next-token logits of the letters from any instruction model. The state is encoded once and shared by all questions. This alone recovers much of Jev’s behaviour, which led one commentator to argue that the novelty is mainly the inference strategy [5], but it is uncalibrated and sensitive to option order.
- **Fine-tuned letter readout.** The same readout on a small model fine-tuned with LoRA on decision data, with one calibration temperature. This is the recipe of most of the best open systems.
- **Trained decision heads.** Jev [6] packs the state and all options into one sequence with a block-causal mask and scores the options with a pointer head. Other projects score each option on its own path and combine them with a set-level head, which makes the answer independent of option order.
- **Bidirectional encoders** (0.15–0.4B parameters) with a classification head over option markers: very cheap, but weaker on long states and multi-step rules.
- **Discrete diffusion:** a diffusion language model reads all answer slots in one denoising step, so the questions of one request can see each other.

None of the systems we surveyed uses early exits or a looped (shared-block) architecture. Both are established for general language models — confidence-based early exits [7, 8, 9] and recursive or looped transformers [10, 11, 12] — and we test both for typed decisions (Step 11).

How basal-1.0 differs. The open systems are English-first and are built on multilingual bases. On our Polish decision test every open 4B system scores between 0.58 and 0.71 (Section 20), and no Polish typed-decision model with an independent evaluation existed when we started. basal-1.0 uses the letter readout of the second design but differs in four ways: a base model and tokenizer built for Polish (Step 2). Training data whose labels are computed, grounded or verified rather than taken from a larger model’s answers (Section 18). An order-averaged readout that averages both option orders, made cheap by a shared-prefix pass (Step 12) and calibration per decision type with confidence thresholds for a target error (Step 10). It does not target the English public benchmark, on which it is weaker by design.

Table 1: The five best systems on JevBench v1.4 (leaderboard of 23 September 2026; score = harmonic mean of intelligence, calibration, speed and cost) [3]. All five are also in our comparison (Section 20; Jev for quality only).

System	Score	Base model	Design
Jev 1.13.0	63.3	undisclosed	Closed; System One API, reinforcement learning for calibrated decisions [1]
JevK5 v0.2	62.0	Qwen3.5-4B	Fine-tuned letter readout (LoRA, one temperature) with distillation from a larger reasoning model on hard decisions checked twice; CUDA graphs [13]
Hopper	59.4	Qwen3.5-4B	Fine-tuned decision model on the same small base [14]
Winnow-12B	55.6	Gemma 4 12B	Fine-tuned decision model on a larger base [15]
reflex-4B	54.0	Qwen3.5-4B	Fine-tuned decision model on the same small base

3 Evaluation protocol

Readout. Options are labelled A, B, C, ... and shown in the model’s chat template. The assistant turn is prefilled with {"answer": " " and we read the next-token logits of the option letters only a softmax over those letters is the decision distribution. We check that each letter is a single token at that position.

Two option orders. Every item is asked in the original and in the reversed option order, because language models favour answers by their position [16]. The served decision is the argmax of the two canonical-order distributions averaged. This is the accuracy reported for the released models and in all comparisons (Section 20, Step 10). The development tables of Steps 5–9 report the mean of the two per-order accuracies, a diagnostic that differs from the served accuracy by a few tenths of a point. The *flip rate* is the share of items whose top answer changes between the orders. Averaging a permutation and its reversal reduces measured order sensitivity. For three or more options it is not a full symmetrisation (the middle option keeps its position), so we do not claim permutation invariance.

Calibration and coverage. The released models are calibrated exactly as they serve: one probability vector per item (the order-averaged distribution), one temperature per decision type fitted by NLL on the calibration split, and applied unchanged to the test split. Expected calibration error (ECE) and negative log-likelihood (NLL) are reported after scaling. *Coverage at 1% / 5% target error* is the share of test decisions whose confidence reaches a threshold chosen on the calibration split, before the test split is scored, so that the error among accepted calibration decisions is at most 1% / 5%. Equal confidences are accepted or rejected together. We report the coverage and the error actually observed on the test split with that frozen threshold. This is the quantity an application cares about: how many decisions it can automate at a given error budget. In the development tables of Steps 5–9 temperatures, coverage and ECE were computed on per-order readouts (two observations per item). Those columns are diagnostics for comparing training variants and are not the calibration of the released models (Step 10).

Numerical precision. In bf16, decisions of some models changed with the composition of the batch (up to 0.37 in probability for a mixture-of-experts model and 0.03 for fine-tuned dense models). All evaluations therefore run in fp32, and every run starts with a check that batched and single-item probabilities agree within 0.02.

Splits. Split guarantees differ by source. Generated Polish data are split before training by *narrative template*, *statute or domain*, so that test items come from templates, statutes and domains the model has never seen. Converted English data follow the source corpus’s own train/test split, and its test part is divided into calibration and test by question identifier because one source case can yield several questions, 79 calibration items shared a state (and 52 a source row) with a test item. They are removed from calibration before fitting, which also removes the one pair of duplicated tasks with contradictory labels. The test split is unchanged, so comparisons with other systems are unaffected.

What the test split is, and is not. The choice between the two v4 training variants, the calibration and the early-exit thresholds were made on development or calibration data only. The v4 test split was, however, consulted repeatedly during development: the families added in v4 (Step 9) respond to regressions first seen on earlier test splits, the decision to keep v4 instead of v5 (Step 13) used the Polish test sets, and the exit policy to time was at first chosen with test accuracy (re-running that choice on the calibration split gives the same policy, Step 11). The test results are therefore results of an adaptive development process on held-out templates, not of a single untouched final evaluationv - a fresh holdout generated after freezing is future work (Section 23).

Table 2: Baseline: accuracy (mean over both orders) with 95% bootstrap intervals, flip rate and ECE.

System	Params	PL knowledge	Gen. knowledge	Decisions	Understanding	EN control	Flip	ECE
Jev 1.13.0 (API)	–	0.789 [0.77,0.81]	0.884 [0.84,0.93]	0.788 [0.75,0.82]	0.855 [0.84,0.87]	0.972 [0.95,0.99]	0.071	0.029
Bielik-11B-v3.0	11B	0.660 [0.63,0.68]	0.667 [0.61,0.72]	0.760 [0.73,0.79]	0.848 [0.83,0.87]	0.945 [0.92,0.97]	0.176	0.151
Bielik-PL-11B-v3.0	11B	0.659 [0.64,0.68]	0.654 [0.60,0.71]	0.750 [0.72,0.78]	0.838 [0.82,0.86]	0.947 [0.92,0.97]	0.185	0.147
Gemma-4-12B-it	12B	0.667 [0.64,0.69]	0.760 [0.70,0.81]	0.766 [0.73,0.80]	0.838 [0.82,0.86]	0.967 [0.94,0.98]	0.157	0.191
Qwen3.5-9B	9B	0.638 [0.61,0.67]	0.677 [0.62,0.73]	0.757 [0.72,0.79]	0.822 [0.80,0.84]	0.962 [0.94,0.98]	0.195	0.089
Gemma-4-E4B-it	E4B	0.586 [0.56,0.61]	0.664 [0.60,0.72]	0.752 [0.72,0.78]	0.810 [0.79,0.83]	0.937 [0.91,0.96]	0.206	0.185
Qwen3.5-4B	4B	0.562 [0.54,0.59]	0.619 [0.56,0.68]	0.723 [0.69,0.75]	0.819 [0.80,0.84]	0.947 [0.92,0.97]	0.235	0.093
Bielik-4.5B-v3.0	4.5B	0.552 [0.53,0.58]	0.530 [0.47,0.59]	0.658 [0.62,0.69]	0.778 [0.75,0.80]	0.915 [0.89,0.94]	0.258	0.226
PLLuM-12B-2512	12B	0.503 [0.48,0.53]	0.490 [0.43,0.55]	0.646 [0.61,0.68]	0.783 [0.76,0.81]	0.908 [0.88,0.94]	0.269	0.128
Bielik-1.5B-v3.0	1.5B	0.465 [0.44,0.49]	0.432 [0.37,0.49]	0.553 [0.52,0.59]	0.557 [0.53,0.58]	0.793 [0.75,0.83]	0.338	0.187
Llama-PLLuM-8B-2512	8B	0.441 [0.42,0.46]	0.447 [0.39,0.50]	0.617 [0.58,0.65]	0.779 [0.76,0.80]	0.885 [0.85,0.92]	0.389	0.089
Qwen3.5-2B	2B	0.386 [0.36,0.41]	0.462 [0.41,0.52]	0.575 [0.54,0.61]	0.741 [0.71,0.76]	0.880 [0.85,0.91]	0.413	0.077

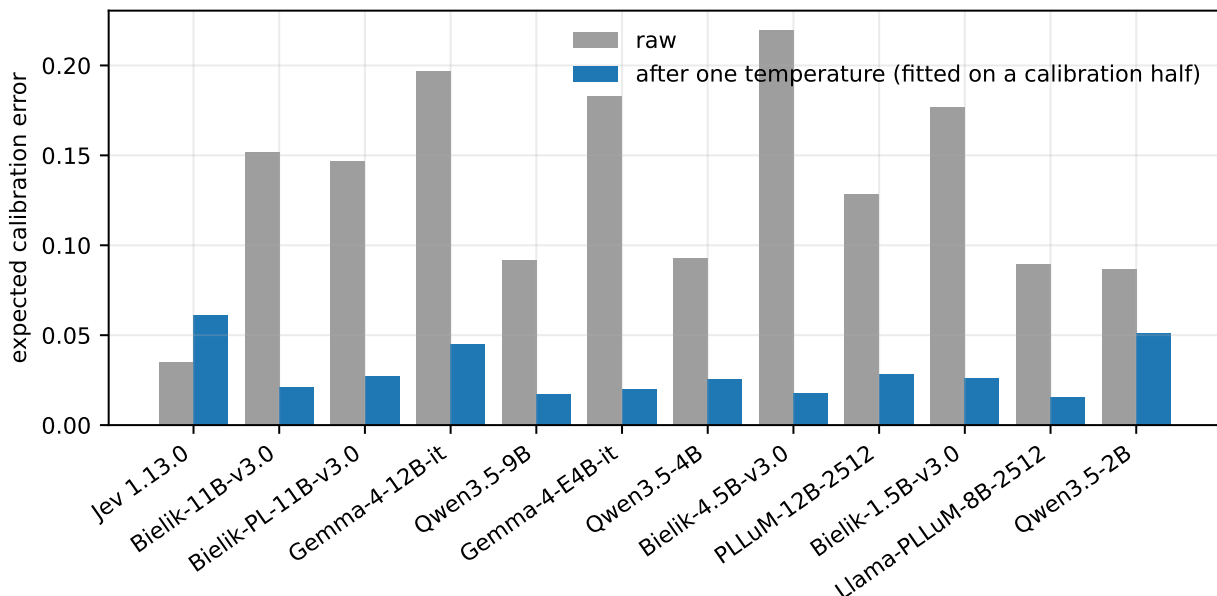


Figure 2: Step 1: calibration error of the baseline systems before and after a single temperature fitted on one half of the baseline benchmark and evaluated on the other half. One scalar removes most of the gap for every open model. Jev is already calibrated as it comes (a fitted temperature improves its NLL but worsens its ECE, 0.035→0.061).

4 Step 1 — Baseline study: where is the gap?

What we did. We built a frozen Polish development suite of 3,079 items (3,076 scored) in five tiers: Polish domain knowledge (exam questions [17, 18, 19]), general knowledge in Polish [20], Polish decisions, Polish understanding (sentiment [21], topic [22], entailment [23], reading [24]) and an English control. We evaluated Jev and eleven open models (1.5B–12B, including Bielik [25] and PLLuM [26]) with the protocol above and recorded an errata file for defective items.

Findings. (Table 2, Figure 2.) The gap to Jev is mainly *knowledge*: 12–13 points on Polish domain knowledge for the best open models. On Polish understanding and synthetic decisions, the best open models are within 2–4 points of Jev, and Bielik-11B is statistically indistinguishable from it on both. Calibration only *looks* like a gap: a single temperature [27] fitted on half of the data brings every open model to ECE of about 0.05 or less. The Polish-specific tokenizer of the Bielik v3 family (APT4) [28] turns the same Polish text into fewer tokens than the tokenizers of the other models (quantified in Step 2).

Table 3: Tokenizer ablation on identical texts. Tokens/word and ratios to APT4 (lower is cheaper). *Rel. throughput*: decisions per second at batch 32 for the mean Polish prompt length of each tokenizer, relative to APT4, using the measured length–time curve of the same 4.5B model (Table 4). The last column is the English ratio: values below 1 mean APT4 needs more tokens on English.

Tokenizer	Vocab	PL decision prompts tok/word	vs APT4	PL text vs APT4	Mean PL tokens	Rel. throughput	EN prompts vs APT4
APT4 (Bielik v3 1.5B/4.5B/PL-11B)	32k	2.11	1.00	1.00	260	1.00	1.00
Bielik-11B v3 (general vocabulary)	32k	3.18	1.51	1.60	393	0.66	0.70
Qwen3.5	248k	2.49	1.18	1.20	307	0.84	0.62
Gemma 4	262k	2.46	1.17	1.19	304	0.85	0.63
Llama 3.1 (Llama-PLLuM-8B)	128k	2.73	1.29	1.39	337	0.77	0.58
Mistral-Nemo (PLLuM-12B)	131k	2.59	1.23	1.26	320	0.80	0.63
LFM2.5	125k	2.49	1.18	1.24	307	0.84	0.60

Table 4: Cost of prompt length on the same 4.5B model (H100, bf16, eager execution, median).

Prompt tokens	Latency, batch 1 (ms)	Decisions/s, batch 32
128	31.8	350
192	32.2	236
256	32.1	176
320	32.0	139
384	31.9	117
512	31.8	88
768	32.0	59
1024	32.2	44
1536	41.5	28
2048	53.6	21

Decision. Build on Bielik v3 [28, 25] (4.5B as the main model, 11B as a stronger variant and teacher), which shares the APT4 tokenizer across sizes. Do not compete on raw calibration error, which is cheap to fix; compete on coverage at a fixed error rate. Address knowledge separately from decisions.

5 Step 2 — Tokenizer ablation: why a Polish-specific tokenizer

Question. The tokenizer fixes how many tokens a Polish state costs. Fewer tokens mean less compute per decision and more room for long documents, but a tokenizer specialised for Polish may cost more on English. We measured both, and then translated token counts into time on the same model.

What we did. (1) We tokenised identical texts with seven tokenizers: 2,000 Polish decision prompts from the test set (state, question and options, without chat template), 2,000 general Polish texts (statutes, court decisions, encyclopaedia), and 1,479 English decision prompts. We report tokens per whitespace word and the ratio to APT4. APT4 is the 32k-token vocabulary of Bielik v3 1.5B, 4.5B and PL-11B, obtained by vocabulary transplant with FOCUS [29]. (2) On one H100 we measured the 4.5B model’s forward pass as a function of prompt length (bf16, batch 1 and batch 32, random tokens, median of repeated runs), and used it to convert each tokenizer’s mean prompt length into decisions per second for the *same* architecture.

Findings. (Tables 3 and 4.) On Polish decision prompts every alternative needs 17–51% more tokens than APT4 (general Polish text: 19–60%). Large multilingual vocabularies (125k–262k tokens) do not close the gap, and the general-vocabulary 11B model of the same family is the most expensive, which isolates the effect of the tokenizer from the model family. The cost of length depends on how the model is served. At batch size 1 the latency is flat up to about 1k tokens (about 32 ms, dominated by per-layer overhead), so for short prompts the tokenizer does not change single-request latency, it does for long documents (1.5k–2k tokens: 42–54 ms). At batch 32, which is how a server runs, throughput falls almost in proportion to length, so the same GPU makes 15–34% fewer decisions per second with the other tokenizers. The 32k vocabulary also keeps the embedding and output matrices small (131M parameters in the 4.5B model, against about 1.0B for a 248k vocabulary at the same width). All tokenizers encode the option letters as single tokens after the answer prefix, so the readout works everywhere. The price is English: APT4 needs about $1.6\times$ more tokens than the multilingual tokenizers on English prompts.

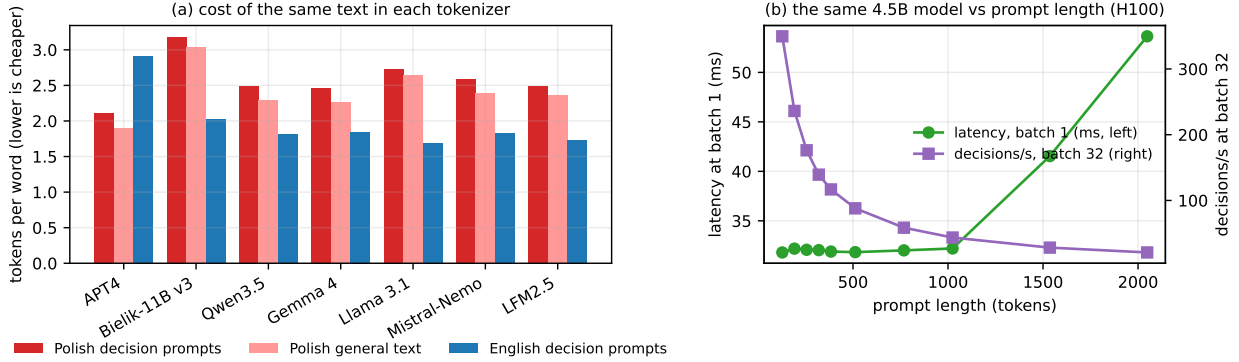


Figure 3: Step 2: (a) tokens per whitespace word for the same texts in each tokenizer (APT4 is the tokenizer of the chosen model family) (b) the same 4.5B model on an H100: latency at batch size 1 is flat up to about 1,000 tokens, while throughput at batch 32 falls roughly in proportion to prompt length — which is where fewer tokens pay off.

Figure 3 shows both measurements side by side.

Decision. Use the APT4 tokenizer. It is the cheapest for Polish at every text type we measured, the other tokenizers make 15–34% fewer decisions per second at server batch sizes, and it is shared by the 1.5B, 4.5B and 11B models, which makes exact teacher–student distillation possible. The English overhead is accepted because Polish is the primary language and English prompts remain within the context budget (only 1.6% of English test prompts exceed 1,024 tokens).

6 Step 3 — Does knowledge distillation on generic text help? (stopped pilot)

What we did. We distilled [30] the full-vocabulary output distribution of Bielik-PL-11B into Bielik-4.5B on Polish web text and Wikipedia (the two models share a tokenizer, so exact logit distillation is possible). The run was planned for 1,145 steps (≈ 150 M tokens). We evaluated it at 50M tokens and stopped it at step 479 (≈ 63 M tokens) on the strength of that early result.

Findings at 50M tokens (baseline benchmark, 3,076 items Table 5). Polish knowledge dropped by 2.0 points (0.552 $\rightarrow$ 0.532), general knowledge by 2.2 points, and Polish decisions and reading comprehension did not change. Raw calibration improved strongly (ECE 0.220 $\rightarrow$ 0.077 on the held-out half), but a single temperature fitted on a calibration half and applied to the *untouched* model gives a better result (ECE 0.018, NLL 0.818 vs 0.037 and 0.845 for the distilled model after its own temperature). The calibration gain of distillation is therefore not a reason to use it.

Table 5: Distillation on generic text, early check at 50M tokens. Accuracy per tier of the baseline benchmark, flip rate, ECE and NLL before and after temperature scaling.

Model	Know.	Gen. kn.	Dec.	Read.	EN	Flip	ECE	ECE _T	NLL	NLL _T
Bielik-PL-11B (teacher)	0.659	0.654	0.750	0.838	0.947	0.185	0.147	0.027	0.990	0.610
Bielik-4.5B (start)	0.552	0.530	0.658	0.778	0.915	0.258	0.220	0.018	1.635	0.818
KD v1, 50M tokens	0.532	0.508	0.657	0.779	0.905	0.270	0.077	0.037	0.876	0.845

Limits of this result. This is a single run, stopped at about 40% of its budget, with one loss (KL to the teacher) and without the planned controls: plain continued training on the same tokens with cross-entropy, and KL plus cross-entropy. It shows that 50M tokens of distillation do not transfer knowledge and that the calibration effect is explained by temperature. It does not show that longer distillation could not help, nor whether distillation *before* decision training would improve the final decision model.

Decision. Stop generic-text distillation and spend the compute on decision data, where every later step showed large, measurable gains. Distillation is used later only at the level of decisions and only where the teacher is demonstrably

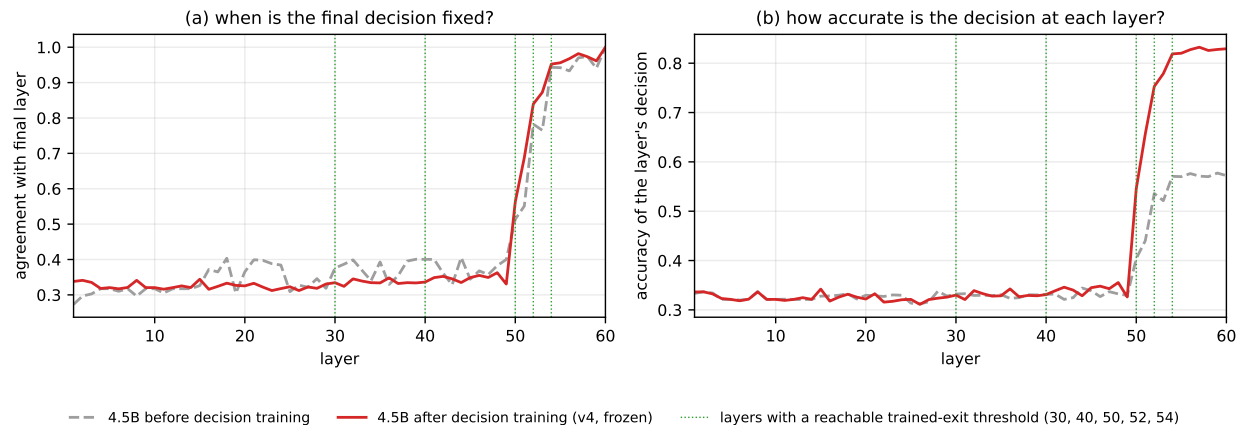


Figure 4: Where the decision becomes readable through the output head of the 4.5B model (60 layers), before and after decision training, on the same stratified readouts (48 families of the v4 test, 2,870 readouts, all option counts). (a) Agreement of each layer’s decision with the final layer’s decision (b) accuracy of each layer’s decision. Dotted vertical lines: layers at which the trained exit heads of Step 11 have a reachable stopping threshold.

better (Step 8). A controlled, full-budget comparison (KL vs cross-entropy vs KL+cross-entropy, followed by decision training) remains open.

7 Step 4 — Where does the decision form? (logit lens)

What we did. We read the option distribution from every intermediate layer of the 4.5B model through the final normalisation and output head, without training any new parameters (the “logit lens” [31]), for the model before decision training and for the frozen v4 model after decision training. For each layer we measure how often its decision agrees with the decision of the last layer, and how accurate it is. The sample is stratified: up to 30 items from each of the 48 families of the v4 test (1,435 items, both option orders, 2,870 readouts, all option counts). As a check of the probe, its final-layer decision equals the decision of the normal readout on 98.8% of the readouts (two-option items 98.8%, three or more options 98.8%).

Findings (Figure 4).

- **Nothing is readable through the output head in the first 49 layers.** Up to layer 49 the accuracy of the intermediate decision is at chance (0.25 for items with three or more options, 0.50 for two options) both before and after decision training.
- **Readability appears in the last ten layers.** After decision training, agreement with the final layer passes 50% at layer 50 and 90% at layer 54, and reaches 99% only at the last layer. Per family, 90% agreement is first reached between layers 50 and 58 (median 53). Before decision training the curve is similar, with a lower final accuracy (0.57 against 0.83). (An earlier version of this analysis, on the first 1,500 test items only, which are dominated by two deadline families, showed a sharper transition at layers 51–53. The stratified sample does not support that.)
- **What this does and does not show.** The logit lens measures when the decision becomes readable through the *unchanged* final head. It does not establish when the information first becomes available in the hidden state, which a trained probe (a tuned lens [32]) may read earlier. Trained exit heads (Step 11) are such probes, and they stop on average at layer 49. For the untrained head, a perfect exit at the layer of 90% agreement (54) would save at most $60/54 = 1.11\times$.

Decision. No untrained early exit. Early exit is revisited with *trained* exit heads after decision training (Step 11), since a trained head may read the decision earlier than the unchanged output head does.

Table 6: Step 5, data v1: test split (unseen templates and statutes, 2,196 items). Rows marked as data v2 are later models evaluated on the same v1 test. Cov@1%/5% = share of decisions accepted at the confidence threshold that keeps calibration error at or below 1%/5%.

System	Acc [95% CI]	Flip	ECE _T	NLL _T	Cov@1%	Cov@5%
Jev 1.13.0 (API)	0.686 [0.668, 0.704]	0.152	0.046	0.625	0.076	0.199
Bielik-PL-11B + aug (data v2)	0.837 [0.821, 0.851]	0.018	0.029	0.280	0.620	0.778
Bielik-4.5B + aug (data v2)	0.813 [0.797, 0.829]	0.029	0.029	0.316	0.579	0.686
Bielik-4.5B + pointer head (data v2)	0.781 [0.763, 0.798]	0.000	0.020	0.433	0.392	0.544
Bielik-4.5B Base, full FT (data v2)	0.777 [0.761, 0.793]	0.112	0.025	0.392	0.396	0.528
Bielik-4.5B + aug_cons (v1)	0.835 [0.820, 0.850]	0.040	0.012	0.286	0.606	0.698
Bielik-4.5B + aug (v1)	0.830 [0.814, 0.845]	0.031	0.027	0.308	0.582	0.704
Bielik-4.5B + sft (v1)	0.825 [0.812, 0.838]	0.164	0.004	0.318	0.556	0.669
Bielik-4.5B (untrained)	0.476 [0.459, 0.495]	0.362	0.067	0.951	0.009	0.055
kev-4B	0.550 [0.533, 0.569]	0.276	0.108	0.878	0.023	0.075
JevK5 (as of 24 Sep.)	0.549 [0.532, 0.567]	0.301	0.085	0.894	0.064	0.091
Qwen3.5-4B	0.493 [0.476, 0.510]	0.368	0.058	0.927	0.055	0.069
LFM2.5-8B-A1B	0.427 [0.414, 0.440]	0.741	0.076	1.080	0.000	0.000

8 Step 5 — First decision training

Data (v1). We chose sources whose labels do not depend on one model’s opinion:

- *Deadlines* (8,000): the last day of an appeal or complaint period under three Polish procedural codes, and whether a filing was on time given how it was submitted. Labels come from a versioned calendar of public holidays and the day-shift rules, checked by hand-computed unit tests. Distractors are typical mistakes (forgetting the weekend shift, counting the delivery day, one week later).
- *Amounts* (8,000): gross amounts, approval thresholds and budget limits, with the rule stated in the state.
- *Statute questions* (1,701): written by an open-weight model from passages of 691 statutes, with a verbatim evidence quote and two independent verifiers.
- Public Polish sentiment and topic data (training splits only).

Each generator has eight narrative templates: five for training and one each for development, calibration and test. Statutes are split by act. We audited shortcuts (for example, the longest option being correct) and fixed them before training.

Training. LoRA (rank 32, all linear layers), one epoch, cross-entropy restricted to the option letters. Three matched variants: fixed option order (*sft*), random order per example (*aug*), and two random orders plus a Jensen–Shannon consistency penalty (*aug_cons*).

Findings. (Table 6.) One epoch on 15k examples lifts the 4.5B model from 0.476 to 0.825–0.835, well above Jev (0.686), and it accepts 56–61% of decisions at 1% error versus 8% for Jev. Random option order cuts the flip rate from 0.164 to 0.031 at equal accuracy. The consistency penalty adds nothing. Open typed-decision models trained in English stay near 0.55. One family, “was the filing on time?”, stayed near chance for every system. On the baseline suite the trained model kept its knowledge and improved understanding, but decisions of types it was not trained on did not improve (0.662 → 0.656).

Decision. Keep random option order in every run. Drop the consistency penalty. The gain is specific to the trained decision types, so the next data iteration must add *breadth* of decision types.

9 Step 6 — A faster base model?

What we did. We evaluated a mixture-of-experts model with about 1B active parameters (LFM2.5-8B-A1B) as a potentially much faster base, including a tokenizer transplant plan.

Findings.

Table 7: Step 7, data v2: test split (5,639 items; two unseen domains, unseen templates and statutes).

System	Acc [95% CI]	Flip	ECE _T	NLL _T	Cov@1%	Cov@5%
Jev 1.13.0 (API)	0.808 [0.799, 0.817]	0.085	0.029	0.399	0.342	0.626
Bielik-PL-11B + aug (data v2)	0.903 [0.896, 0.910]	0.018	0.016	0.194	0.767	0.904
Bielik-4.5B + aug (data v2)	0.876 [0.867, 0.884]	0.036	0.011	0.255	0.669	0.812
Bielik-4.5B + pointer head (data v2)	0.860 [0.851, 0.869]	0.000	0.011	0.316	0.574	0.729
Bielik-4.5B Base, full FT (data v2)	0.823 [0.815, 0.832]	0.089	0.013	0.367	0.448	0.660
Bielik-PL-11B (untrained)	0.650 [0.640, 0.661]	0.283	0.026	0.719	0.010	0.118
Bielik-4.5B (untrained)	0.593 [0.581, 0.604]	0.240	0.035	0.842	0.006	0.009

 Table 8: Data v2 test by item group. *Computed*: deadlines and amounts; *Statutes*: grounded questions; *Broad*: base and counterfactual items of the broad types (unseen domains); *Suff. yes/no*: “does the material allow a decision?” with the deciding evidence present / removed.

System	Computed	Statutes	Broad	Suff. yes	Suff. no
Jev 1.13.0 (API)	0.686	0.990	0.959	0.996	0.815
Bielik-PL-11B + aug	0.863	0.985	0.935	0.938	0.963
Bielik-4.5B + aug	0.840	0.974	0.893	0.902	0.950
Bielik-4.5B + pointer	0.811	0.985	0.888	0.917	0.927
Bielik-4.5B Base, full FT	0.820	0.959	0.824	0.967	0.740

- **Weak Polish.** Polish knowledge 0.362 (Bielik-4.5B: 0.550), a medical exam at 0.315 (chance 0.20) and Polish reading comprehension (Belebele [24]) at 0.633 against 0.86–0.90 for the other candidates. The gap is missing Polish in the model, not the tokenizer, so a tokenizer transplant would not close it.
- **Position instead of content.** Without training, the answer changes in 74% of questions when the options are reversed (flip rate 0.74): the model chooses positions, not options.
- **The router must run in fp32.** In bf16 the readout depended on the *other* prompts in the batch: the letter probabilities of one prompt changed by up to 0.37 even in a batch of identical prompts, while in fp32 the difference was exactly 0. Rounding noise of bf16 flips the choice of experts made by the mixture-of-experts router, and a different set of experts gives a different answer. Its first bf16 result (0.411) was therefore discarded and the model was evaluated in fp32 (0.427 on the Polish decision test). For serving this is a hard limit: the router (and in practice the whole forward pass) has to stay in fp32, which gives up the speed that a 1B-active model was supposed to bring, and it rules out the bf16, FP8 and NVFP4 paths, CUDA graphs over batches of mixed requests and the batch-invariant results on which our serving stack relies (Step 12). A mixture-of-experts base becomes attractive only with a router that is stable in low precision.

After this finding every readout in the project runs a parity check (the same prompt alone and inside a batch) and is refused if the probabilities differ by more than 0.02.

Decision. Reject it as a base. Speed will come from the serving stack, early exits and compression of a model that is already good in Polish (Steps 11 and 12).

10 Step 7 — Breadth, evidence sufficiency, and design ablations

Data (v2). We added twelve broad decision types (routing, policy compliance, violation, complaint handling, document type, urgency, risk, eligibility, personal data, satisfaction, next step, offer selection) across fourteen domains and seven writing registers. Every generated case yields up to four items: a base item a *counterfactual* in which a minimal change of the deciding fact changes the answer and two *evidence-sufficiency* questions (“does the material allow a decision?”), one with and one without the deciding evidence. A case is kept only if two verifier models, different from the generator and answering without seeing the intended label, agree with it. 5,613 of 15,000 cases (37%) passed, giving 18,424 items (base items, and their companions where these passed as well). Splits are by domain: two domains appear only in the test split. With 12k deadline and 10k amount items, v2 has 31,653 training and 5,639 test items.

Findings. (Tables 7 and 8.) The 4.5B model reaches 0.876 against 0.808 for Jev and accepts 67% of decisions at 1% error (Jev: 34%). The 11B model reaches 0.903 with 77%. The lead comes from code-computed decisions and from recognising insufficient evidence (0.95–0.96 vs 0.82. Jev too often says the material is sufficient). Jev remains better on

Table 9: Step 8, data v3 (7,118 test items, 1,479 in English): distillation versus cross-entropy only.

System	Acc [95% CI]	Flip	NLL _T	Cov@1%	Cov@5%
Jev 1.13.0 (API)	0.792 [0.783, 0.800]	0.085	0.485	0.248	0.569
11B, trained on v2 (teacher)	0.848 [0.839, 0.856]	0.057	0.340	0.547	0.757
4.5B, v3, CE	0.852 [0.844, 0.860]	0.066	0.322	0.598	0.749
4.5B, v3, CE + KD	0.843 [0.835, 0.852]	0.058	0.330	0.605	0.713

System	Polish	English
Jev 1.13.0 (API)	0.817	0.736
11B, trained on v2 (teacher)	0.904	0.641
4.5B, v3, CE	0.883	0.756
4.5B, v3, CE + KD	0.880	0.724

broad decision types in unseen domains. Because broad items were kept only when two verifiers agreed, they are easy by construction. They measure coverage of decision types, not difficulty.

Ablations on data v2.

- **Learning rate.** At 10^{-4} all runs on v2 diverged (loss collapse or NaN), although this rate was stable on v1. At 5×10^{-5} every run was stable. The trainer now skips steps with non-finite loss or gradient.
- **Pointer head instead of letters.** We implemented a head that scores each option in its own segment, with shared position ids and a block attention mask, so the answer cannot depend on option order (flip exactly 0, verified by unit tests). It is less accurate (0.860 vs 0.876), gives lower coverage (0.574 vs 0.669 at 1%) and cannot compare options with each other: choosing the last day among candidate dates falls to 0.144, because each option sees only itself.
- **Full fine-tuning of the base (pretrained) model** with AdamW applied directly to bf16 weights at 10^{-5} , a recipe reported to preserve base knowledge at a much larger data scale [33]: 0.823 vs 0.876 for LoRA on the instruction model, lower coverage, higher flip, and no knowledge gain (0.553 vs 0.559).
- **Scale.** The same recipe on the 11B model gains 2.7 points and keeps 9 points more Polish knowledge.
- **Per-type temperatures** (separate for choice, noul, score) changed coverage by less than 0.01.

Decomposition. “Was the filing on time?” remained near chance for all systems. The state does not state the length of the period, so one forward pass must recall the rule, count days with the holiday shift and apply the rule for the submission mode. The trained model answered “no” in 79% of such items. Asking two elementary questions about the same state (“what is the last day of the period?”, “what is the legal filing date?”) and comparing the answers in code raised accuracy from 0.48 to 0.92 for the 4.5B model and from 0.52 to 0.69 for Jev (Table 11, Figure 6).

Decision. Letter readout with order augmentation remains the default (a pointer head is reserved for independent labels such as routing). LoRA on the instruction model at 5×10^{-5} . Composite rules are decomposed into typed primitives and combined by the application. The benchmark and interface expose this pattern.

11 Step 8 — Bilingual data and decision distillation

Data (v3). To keep English ability, we added 13,500 English decision items converted from a public typed-decision corpus [34] (about 30% of training), after fixing its biases (the correct option was listed first in 39–65% of questions, yes/no labels were 70% “no”), capping any domain at 12% and removing any state that also occurs in our test sets. The test split gains 1,479 English items.

Decision distillation. The student is trained with $(1 - \alpha) \text{CE} + \alpha \text{KL}(p_T \| p_S)$ over the option distribution, where p_T is the teacher’s distribution averaged over both orders ($\alpha = 0.5$). The teacher was the 11B model trained on v2. A matched control uses cross-entropy only.

Findings. (Table 9.) English data did not hurt Polish (0.876→0.883) and made the 4.5B model slightly better than Jev on English decisions (0.756 vs 0.736). Polish knowledge rose for the first time (0.559→0.576). Distillation lowered overall accuracy slightly (0.843 vs 0.852) and hurt English (0.724 vs 0.756), because the teacher had never seen English

Table 10: Step 9, data v4 (8,560 test items, corrected notice-period labels). Development diagnostics on per-order readouts: accuracy is the mean of the two per-order accuracies, and NLL_T and coverage use one temperature fitted on the two order readouts separately. The served predictor is evaluated in Table 13. Bottom: accuracy by item group (*Rules*: the ten new families).

System	Acc [95% CI]	Flip	NLL_T	Cov@1%	Cov@5%
Jev 1.13.0 (API)	0.765 [0.756, 0.773]	0.108	0.540	0.211	0.489
11B, v4 (teacher)	0.886 [0.879, 0.892]	0.036	0.268	0.654	0.806
4.5B, v4, CE	0.853 [0.846, 0.860]	0.058	0.334	0.533	0.733
4.5B, v4, CE + KD (teacher-correct)	0.852 [0.845, 0.859]	0.051	0.335	0.560	0.723

System	Computed	Rules	Statutes	Broad	English
Jev 1.13.0 (API)	0.686	0.635	0.990	0.938	0.736
11B, v4 (teacher)	0.861	0.942	0.979	0.936	0.787
4.5B, v4, CE	0.851	0.921	0.979	0.890	0.738
4.5B, v4, CE + KD (teacher-correct)	0.848	0.916	0.974	0.894	0.741

Table 11: Decomposition of “was the filing on time?” across iterations (v2 test, 494 composite questions). The decomposed answer is computed in code as filing date $\leq$ last day.

System	Last day	Filing date	Decomposed
Jev 1.13.0 (API)	0.306	0.658	0.688
4.5B, v2	0.850	1.000	0.921
4.5B, v3, CE	0.739	1.000	0.875
4.5B, v3, CE + KD	0.838	1.000	0.939
4.5B, v4, CE	0.998	1.000	1.000
11B, v2	0.998	1.000	1.000
11B, v4	1.000	1.000	1.000

(0.641) but it clearly helped where the teacher was strong: the last-day primitive improved from 0.739 to 0.838 and the decomposed filing decision from 0.875 to 0.939.

Decision. Distil only from a teacher that is strong in the target domain: retrain the teacher on the bilingual data, and apply the distillation term only on items where the teacher’s top answer is correct.

12 Step 9 — Rule families with twins and targeted data

Data (v4). Ten new Polish rule families with labels computed by code, each generated as a *twin pair*: the second item changes one fact or one rule and thereby changes the correct answer (pairs that do not flip are discarded. Both twins stay in one split). Where a rule would otherwise have to be recalled (court fees, income tax, interest for late payment, price lists, contractual working-day periods) it is stated in the state, so the label follows from the evidence. Where it is well-established law, the model must know it: the labour-court appeal period, the statutory notice period by length of employment, the limitation period of claims, and the consumer withdrawal period. One family combines a table in the state with the holiday calendar (a duty roster). We also added 6,000 last-day items and 4,962 decomposition primitives built from the training-split filing questions, the exact format in which v3 had regressed. v4 has 63,663 training and 8,560 test items (1,442 from the new families). Both the teacher (11B) and the student are retrained on v4.

Findings. (Tables 10 and 11.) The retrained teacher now handles English (0.787, previously 0.641). On the ten new rule families the 4.5B model reaches 0.921 and Jev 0.635. Over the whole v4 test the 4.5B model scores 0.853 against 0.765 and accepts 53% of per-order readouts at 1% error against 21% (development diagnostics. The served predictor is evaluated in Step 10). On the notice-period family the model reproduces an error of the generator, which ignored seniority reached during the notice period (Section 18): all 18 affected test items are answered with the old, incorrect label. The targeted data restored the last-day primitive (0.998) and the decomposed filing decision (1.000). The weakest new families are the consumer withdrawal period (0.64), which requires knowing both the period and that sending the notice in time is sufficient, and tiered price lists (0.82). The distilled 4.5B variant (teacher-correct distillation) reaches 0.852 on the test split. It was slightly worse on the calibration split (0.844 vs 0.847) and was not selected (Step 10).

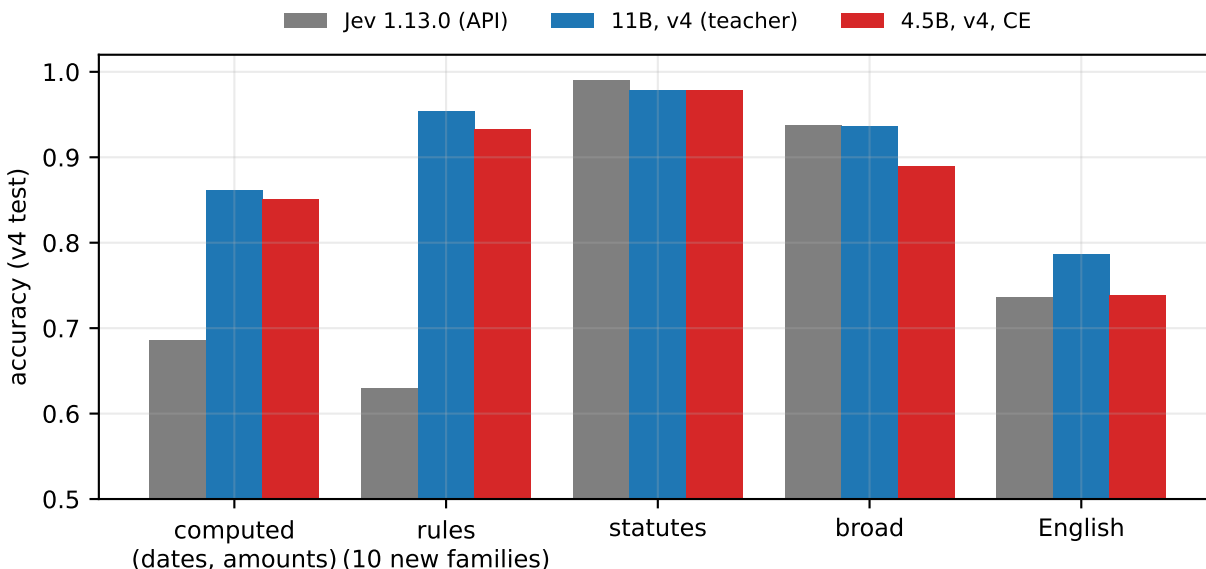


Figure 5: Step 9: accuracy on the v4 test by item group for Jev, the 11B teacher and the 4.5B model. The largest gains are on computed decisions and on the ten rule families with twins; statutes are saturated; English is at Jev’s level.

Table 12: Baseline suite across iterations (accuracy; decisions = types not trained on). All rows, including the untrained models, come from one re-run with the same readout; the untrained values therefore differ from Table 2 by up to 0.4 points.

System	PL knowledge	Gen. knowledge	Decisions	Understanding	EN control	Flip
Jev 1.13.0 (API)	0.789	0.884	0.788	0.855	0.972	0.071
4.5B, untrained	0.550	0.530	0.662	0.780	0.917	0.259
4.5B, v1	0.561	0.553	0.656	0.873	0.943	0.209
4.5B, v2	0.559	0.553	0.675	0.860	0.933	0.210
4.5B, v3, CE	0.576	0.588	0.667	0.861	0.935	0.201
4.5B, v3, CE + KD	0.573	0.561	0.648	0.865	0.933	0.198
4.5B, v4, CE	0.579	0.581	0.698	0.865	0.932	0.196
11B, v2	0.649	0.649	0.750	0.864	0.942	0.182
11B, v4	0.669	0.654	0.772	0.870	0.955	0.150

Knowledge across iterations. (Table 12.) Decision training never damaged knowledge, and decisions of untrained types improved to 0.698 for the 4.5B model. Polish knowledge moves only slightly with data (0.550 to 0.579) but clearly with model size (11B: 0.669), consistent with public typed-decision leaderboards where knowledge scales with the base model [35].

13 Step 10 — Freezing a model version

Selection. The released version is selected among the v4 candidates by accuracy on the calibration split (ties: coverage at 1% error). The test split is not used for this choice. The 4.5B model trained with cross-entropy on v4 (calibration accuracy 0.847) was preferred over its distilled twin (0.844).

Calibration of the served predictor. The engine averages the two option orders and then applies one temperature per decision type, so the released calibration is fitted on exactly that predictor: one order-averaged probability vector per calibration item, a temperature per type by NLL (choice 1.07, yes/no 1.48, score 1.11 for the 4.5B model), and confidence thresholds for 1% and 5% accepted error chosen on the calibration split with equal confidences kept together. The thresholds are then applied once, frozen, to the test split (Table 13). The same procedure is applied to Jev’s calibration and test readouts. With the threshold fixed in advance, the 4.5B model accepts 58.6% of test decisions

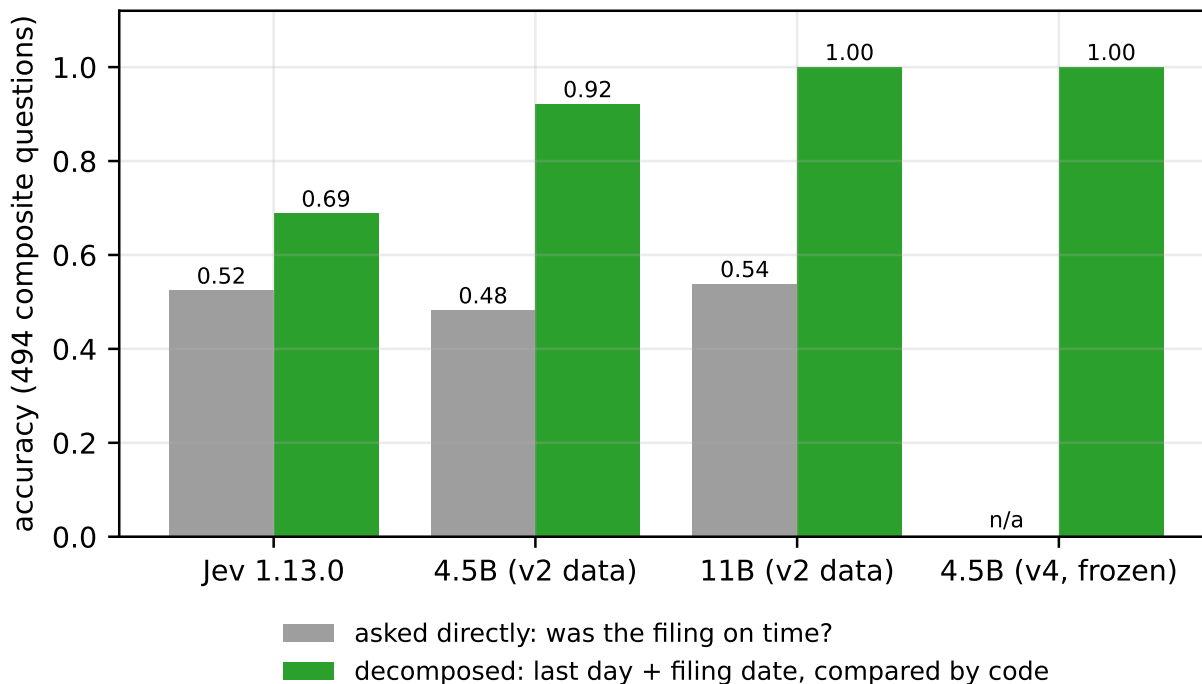


Figure 6: Steps 7 and 9: “was the filing on time?” asked directly versus decomposed into two typed primitives (the last day of the period and the filing date) whose answers are compared by code; 494 composite questions of the v2 test. Asked directly on the 494 composite questions of the v4 test, the frozen v4 model scores 0.476 (it answers “not on time” almost always), so the composite question is served only through the decomposition (decomposed: 1.000, Table 11).

Table 13: Step 10: the served predictor on the v4 test (8,560 items; corrected notice-period labels, Section 18). Accuracy of the order-averaged decision, ECE and NLL after the per-type temperatures, and coverage (with the error observed among accepted decisions) at confidence thresholds fixed on the calibration split before testing. Last column: the best coverage at 1% error read off the test curve itself (retrospective, not an operating point). fp32 replay of saved predictions; the bf16 engine agrees with fp32 on 99.3–99.7% of decisions (Step 12).

System	Acc.	ECE	NLL	Cov.@1% (obs. err.)	Cov.@5% (obs. err.)	Retro.@1%
basal-1.0-4.5B	0.858	0.010	0.321	58.6% (1.22%)	75.0% (4.41%)	56.2%
basal-1.0-1.5B	0.829	0.004	0.383	49.5% (1.37%)	68.6% (5.55%)	45.2%
Jev 1.13.0 (API)	0.772	0.014	0.514	18.1% (0.26%)	49.0% (4.00%)	29.7%

at a target error of 1%. The error actually observed among them is 1.2% (61 of 5,013), slightly above the target, and 75.0% at a target of 5% (observed 4.4%). Under the same procedure Jev accepts 18.1% (observed 0.3%) and 49.0% (4.0%). The earlier release metadata had fitted temperatures and thresholds on the two order readouts separately (two observations per item), which is not the served predictor. The values here replace them in version 1.0.1 of the calibration files, with unchanged weights. These temperatures, thresholds and error rates were fitted and measured on descriptions-only prompts, the format of the evaluation data, which the engine reproduces with "option_keys": "hide". For requests in the default mode, which also shows the option keys, they are not validated (on the public English benchmark in that mode the calibration error is 0.085 for the 4.5B model, Section 20).

Calibration manifest. Each release stores the per-type temperatures, the confidence thresholds for 1% and 5% accepted error with their frozen test outcomes, and a manifest that records the base model, training configuration, data hashes, readout protocol and test metrics. The server applies the per-type temperatures to every answer. The thresholds are for the application to apply. The FP8 and NVFP4 checkpoints carry the same temperatures marked as not validated for them, and no test claims, because their predictions differ.

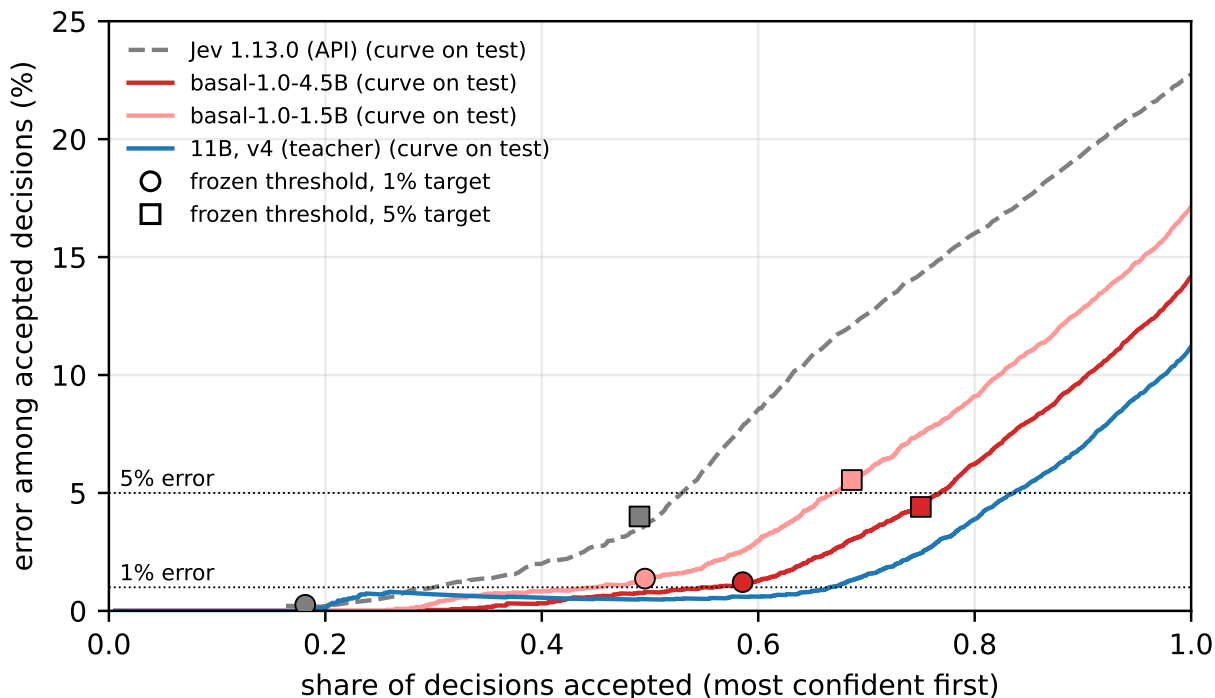


Figure 7: Step 10: error among accepted decisions against the share of decisions accepted, most confident first, on the v4 test (8,560 items, both option orders averaged, points only where the confidence changes). The curves are retrospective: read off the test curve itself, a 1% error budget would allow the 4.5B model 56% and Jev 30%. Markers are the operating points actually obtained with thresholds fixed on the calibration split (Table 13).

Decision. Freeze the cross-entropy 4.5B v4 model with per-type temperatures and release it as **basal-1.0-4.5B**. All later speed work must reproduce its decisions. We check this with argmax agreement against the fp32 reference on 500 test items.

14 Step 11 — Trained early exits and compression

Trained early exits. Small heads (a normalisation layer and a low-rank adapter in front of the frozen output head) are trained at layers 30–55 to match the final layer’s option distribution (KL plus a small cross-entropy term), and a confidence gate with a per-layer threshold chosen on the calibration split decides where to stop. The target was $\geq 1.5\times$ fewer layers without loss. An earlier untrained probe had suggested $1.49\times$ at -0.2 points. That figure is an artefact of its sample. A third of its readouts were the directly asked composite deadline question (“was the filing on time?”), which the frozen model itself cannot answer: its final layer, like the normal readout, picks the same option for almost every readout (984 of 986 in the normal readout, accuracy 0.48). The model is at chance on these items at every depth, so stopping them at layer 10 cost nothing measurable. The composite question is served through the decomposition of Step 7 instead. **Result:** requiring 98% agreement with the final layer on the calibration split, the gate stops on average at layer 49.1 of 60 ($1.22\times$ fewer layers) with test accuracy 0.855 (the full model on the same bf16 readout: 0.853, both scored before the label correction of Section 18). The agreement level to time was first chosen by test accuracy. Choosing it on the calibration split alone (largest calibration depth reduction, every level meeting its agreement target there by construction) selects the same level; latency at batch size 1 improves from 31.5 to 27.8 ms ($1.13\times$). At this level reachable thresholds exist at layers 30 and 40 (confidence thresholds 0.98 and 0.985) and 50, 52 and 54, the heads at 35, 45, 51, 53 and 55 never reach 98% agreement on the calibration split. Stricter agreement levels (99–99.9%) give 1.15 – $1.19\times$.

Interim conclusion. The target of $1.5\times$ is not met: with these heads and data, no exit much before layer 50 reaches the required agreement, consistent with the late readability of Step 4. Early exits also interact badly with batching (a

Table 14: Early exits on the optimised serving path, one loaded model, setting chosen per request (H100, 500 test items; a separate run from Table 17, whose same configuration measured 12.5 instead of 12.4 ms). Latency of one decision at batch size 1 with two / one option orders (ms); two-order decisions per second at 32 option-order passes per forward; accuracy; argmax agreement with the fp32 reference; share of batches stopped before the last layer.

early_exit	2 orders	1 order	Dec./s	Acc.	Agree	Early
No exit heads (reference fast path)	12.4	11.6	62.8	0.826	0.993	0%
off (heads loaded, always final layer)	12.2	11.9	62.5	0.820	0.994	0%
0.999	12.8	10.5	61.1	0.820	0.994	24%
0.995	11.0	10.2	61.7	0.820	0.993	33%
0.99	10.5	10.2	64.5	0.820	0.992	47%
0.98	10.9	10.5	67.8	0.828	0.982	79%

batch can stop only when all its rows stop) and with CUDA graphs (the graph must be split at every exit). We therefore optimised the serving path first (Step 12) and then measured early exits on top of it (below).

Early exits in the optimised server, chosen per request. Early exit only pays if it survives the serving path of Step 12. We split the compiled CUDA-graph forward into segments at the exit layers. After each segment the exit heads score the answer letters at the readout positions and the batch stops when every readout is above the threshold of that layer (a batch can only stop as a whole). The server loads the heads in the mode `fast-exit`. Because the thresholds for all agreement levels come from the same calibration run, one loaded model can serve all of them: a request may set `early_exit` to `off` (always the final layer) or to an agreement level, and the server batches requests with the same setting together. Table 14 shows the resulting dial. At 0.99 a two-order decision takes 10.5 ms instead of 12.2 ms ($1.16\times$) with agreement to fp32 essentially unchanged (0.992 vs 0.994); at 0.98 throughput is highest but 1.8% of decisions change. Loading the heads and the segmented graphs costs about 7 GB of extra memory.

Decision. Ship the early-exit heads as an optional, per-request speed setting (default `off`; 0.99 recommended when latency matters), not as a fixed part of the model. The gain is real but small: the trained heads reach the required agreement only in the last ten layers (Steps 4 and 11).

Compression to $\approx 2\text{--}3\text{B}$. Four models (the looped one evaluated at one to three loops) are trained the same way (full training, the same data, and distillation from the frozen 4.5B model) and compared on quality, measured latency and memory:

- a *looped* model: layers 1–4 and 57–60 kept, layers 53–56 dropped, and the 48 layers 5–52 replaced by one shared 16-layer block initialised as the average of the three 16-layer groups (5–20, 21–36, 37–52) and applied 1–3 times, with a separate low-rank adapter per loop initialised from the singular value decomposition of each group’s residual (2.14B parameters including the adapters. Trained with a random number of loops so that one model serves three depths);
- a smaller model of the same family (1.5B);
- the 4.5B model pruned to 26 layers (first and last 13, 2.14B) and to 36 layers (first and last 18, $\approx 2.9\text{B}$), following layer-pruning work that removes middle layers [36, 37].

A pipeline check with untrained models already shows a structural point: with three loops the looped model executes 56 layers plus the loop adapters and is *slower* than the full 60-layer model (43.0 vs 31.6 ms at batch size 1, bf16), while using half the memory (4.7 vs 10.1 GB). It can only win on latency at one loop. The looped model is kept only if it beats both baselines on the measured quality–latency–memory trade-off.

Compression: results (Table 15, Figure 8b).

- **The 1.5B model of the same family** is the best compressed model by a wide margin: -2.9 points overall (-3.4 on Polish, -1.0 on English) for $1.7\times$ lower latency, $2.5\times$ the throughput and half the memory. It is trained by full fine-tuning with *decision* distillation (not the generic-text, full-vocabulary distillation of Step 3): the loss is $0.5 \text{CE}(\text{gold}) + 0.5 \text{KL}(p_{\text{teacher}} \parallel p_{\text{student}})$ over the option letters, where p_{teacher} is the frozen 4.5B model’s order-averaged option distribution on each training item, and the option order is shuffled per item. Two epochs of v4 (1,990 steps of 64 items on 8 GPUs), AdamW ($\beta = 0.9, 0.95$, no weight decay), learning rate 2×10^{-5} with a 20-step warm-up and cosine decay to 10%, maximum length 1,536 tokens. The pruned and looped variants below use exactly the same recipe and budget.

Table 15: Compression. Accuracy on the v4 test (8,560 items, fp32; argmax of the probabilities averaged over both option orders, so the frozen model scores 0.858 here and 0.853 under the per-order protocol of Table 10), overall and for Polish/English items; parameters (B), median latency of one prompt (one option order) at batch size 1 and prompts per second at batch 32 (bf16, eager, one H100), peak memory (GB). All smaller variants are trained on the same data with distillation from the frozen model.

Model	Params	Acc.	PL	EN	p50 (ms)	Prompts/s	Mem.
basal-1.0-4.5B (60 layers, frozen)	4.76	0.858	0.883	0.738	31.8	45	14.8
basal-1.0-1.5B (1.5B, distilled)	1.60	0.829	0.849	0.728	19.0	115	7.1
4.5B pruned to 36 layers, distilled	2.91	0.777	0.795	0.690	19.7	74	10.3
4.5B pruned to 26 layers, distilled	2.14	0.547	0.546	0.549	14.5	101	8.5
Looped $\approx 2B$, 1 loop	2.14	0.355	0.363	0.321	17.9	95	8.3
Looped $\approx 2B$, 2 loops	2.14	0.357	0.363	0.329	33.2	57	8.3
Looped $\approx 2B$, 3 loops	2.14	0.356	0.362	0.327	43.7	41	8.3

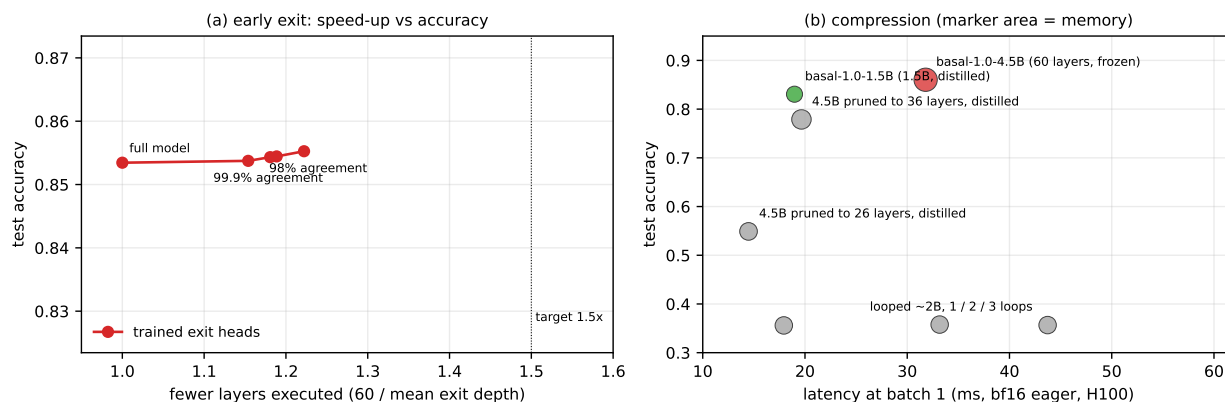


Figure 8: Step 11: (a) trained early exits — test accuracy against the reduction in executed layers for agreement levels 98–99.9% (the target of $1.5\times$ is not reached); (b) compressed models — accuracy against latency at batch size 1, marker area proportional to peak memory.

- **Pruning** keeps the speed but not the quality: 36 layers lose 8.1 points, 26 layers fall to 0.549. Removing whole middle layers from a trained model destroys more than two epochs of distillation can restore, whereas a model pretrained at small size has learned to use its depth.
- **The looped model did not train.** Its training accuracy stayed near 0.4 and at evaluation it puts only 1.3–1.6% of the probability on the answer letters. A parity check rules out an evaluation fault: a save/load round trip and the training-mode and evaluation-mode forward passes give identical outputs, and the number of loops does change the output. The cause is the initialisation: replacing 48 trained layers by one averaged block destroys the network (probability on the answer letters falls from 0.998 for the teacher to 10^{-4} at one loop and 4×10^{-6} at three loops before training), and per-loop low-rank adapters trained on $\approx 30M$ tokens of decision data cannot rebuild it. Published looped models [10] recover with continued pretraining on billions of tokens. Even a working looped model would be slower than the full model at three loops (pipeline check above), and at one loop it would have to beat the 1.5B model at equal latency.

Decision. Keep the 4.5B model as the main model. Offer the distilled 1.5B model as a *lite* variant for throughput-bound or memory-bound deployments (-2.9 points, $2.5\times$ throughput). Drop pruning and the looped model.

Speed of the 1.5B model across GPUs. The compression comparison above used the eager bf16 path on one H100. For the released lite model we measured the optimised serving path on every GPU class with the released benchmark tool (the same 500 test items as Table 26; Table 16).

- **The 1.5B model is $1.9\text{--}2.7\times$ faster than the 4.5B model on every card where both were measured,** with decisions as stable as the 4.5B model’s (agreement with fp32 0.992–0.998). The gain is smallest on the B300

Table 16: basal-1.0-1.5B on the optimised serving path (both option orders, batch size 1, decisions per second with 32 option-order passes per batch). Agreement with fp32 on the same machine (RTX 4090: bf16 is the reference and no agreement was measured, because the fp32 reference run ran out of memory at batch 32 on the 24 GB card). Last two columns: bf16 latency of the 4.5B model on the same card and the speed-up of the 1.5B model. Accuracy on this 500-item speed sample is not a quality measure (in bf16 and FP8 both models are within ± 1 point of 0.83 on it), quality is measured on the full test (Table 15).

GPU	Class	bf16			FP8			4.5B bf16	
		ms	dec./s	agree	ms	dec./s	agree	ms	1.5B speed-up
B300 SXM6	server (Blackwell)	4.7	250	0.994	5.2	224	0.966	8.8	1.9×
H100 80GB	server	6.2	157	0.992	6.3	177	0.964	12.5	2.0×
RTX PRO 6000	workstation	8.8	101	0.998	7.6	128	0.968	19.1	2.2×
RTX 5090	consumer	12.7	67	0.996	9.3	99	0.960	28.0	2.2×
RTX 4090	consumer	12.5	51	ref.	—	—	—	—	—
DGX Spark (GB10)	desktop	34.3	20	0.996	18.1	31	0.962	92.0	2.7×

(1.9×), where both models are limited by kernel launches, and largest on the memory-bound DGX Spark (2.7×), where latency scales with the bytes of weights read per pass.

- **Fastest configurations:** 4.7 ms and 250 decisions/s on a B300, 6.2 ms on an H100 (7.7 ms and 147 decisions/s end to end over HTTP, measured like the systems in Table 24); under 10 ms on the RTX PRO 6000 and, with FP8, on the RTX 5090, 18.1 ms with FP8 on the DGX Spark.
- **FP8** follows the same pattern as for the 4.5B model: no gain on server GPUs, 1.2× on the RTX PRO 6000, 1.4× on the RTX 5090 and 1.9× on the DGX Spark, changing 3–4% of decisions. **NVFP4** again costs quality (DGX Spark: agreement 0.87, accuracy 0.764 against 0.832 in fp32) and is not recommended.
- **RTX 4090:** bf16 only (12.5 ms, 51 decisions/s). The fp32 reference run ran out of memory at batch 32 on the 24 GB card, and compilation of the FP8 path stalled on it, so FP8 and the 4.5B model were not measured on this card.

Reproducibility of the published speeds. The released engine was installed from its repository and the models downloaded from the model hub on fresh cloud machines. The 4.5B numbers reproduce the earlier measurements within 3% (RTX PRO 6000: 19.1 vs 18.9 ms in bf16, 15.2 vs 14.9 ms in FP8; RTX 5090: 28.0 vs 27.3 ms and 19.5 vs 19.4 ms; H100 over HTTP: 14.2 vs 14.1 ms), and a fresh installation returns the same probabilities as the release run (maximum difference 0.01). One rented RTX 5090 was about 3× slower than a second one of the same type (a throttled host); cloud measurements should therefore be repeated on a second machine before they are trusted.

15 Step 12 — Serving: from 177 ms to 12.5 ms per decision

A decision needs one forward pass per option order and no generated tokens, so its latency is the latency of a *prefill*. At batch size 1 a 4.5B model on a 400-token prompt performs about 4 TFLOP, a few milliseconds of H100 compute. The measured 32 ms of a plain bf16 PyTorch forward is mostly the cost of launching thousands of small kernels one by one. We therefore optimised the serving path, not the model, and after every change checked that decisions agree with the fp32 reference (Table 17, Figure 9).

1. bf16 instead of fp32. 176.5 → 32.9 ms per two-order decision (105.7 → 32.4 ms for one order) with 99.7% agreement. Evaluation stays in fp32 (Section 3); serving uses bf16.

2. CUDA graphs. A CUDA graph records the whole forward pass once per input shape and replays it with a single launch. Inputs are padded into a small set of static shapes (batch 1, 2, 4, ..., 32; length 128 to 3072 in steps). Our first implementation padded on the left and passed an explicit attention mask; with several captured graphs it returned almost constant outputs, although each graph was correct on its own. We did not find the root cause (allocator settings, the mask used at capture and the attention implementation were all ruled out). The working design, taken from the decider-4b server [33], pads on the *right* and uses plain causal attention without a mask: padding after the last real token cannot influence it, so we read the hidden state at the last real position. All graphs share one memory pool. Result: 16.5 ms per option order at batch size 1 (23.3 ms for both orders), agreement 99.5%.

Table 17: Serving ladder for the frozen 4.5B model on 500 test items (offline, no HTTP). *1 order / 2 orders*: median latency of one decision at batch size 1 with one option order or with both orders (ms). *Dec./s*: full two-order decisions per second when 32 option-order passes are processed together. *Agree*: argmax agreement with the reference (H100 and RTX PRO 6000: fp32; RTX 5090: bf16 eager on the same GPU). Memory: peak GB.

Serving path	1 order	2 orders	Dec./s	Acc.	Agree	Mem.
<i>H100 80GB</i>						
Eager, fp32 (reference)	105.7	176.5	3.2	0.820	ref.	36.6
Eager, bf16	32.4	32.9	25.1	0.826	0.997	18.4
CUDA graphs	16.5	23.3	30.7	0.822	0.995	11.6
CUDA graphs + shared prefix	16.6	17.1	49.0	0.820	0.995	12.0
Compiled + CUDA graphs	11.6	16.5	40.1	0.826	0.996	11.6
Compiled + graphs + shared prefix	11.6	12.5	62.8	0.826	0.993	12.0
CUDA graphs, FP8 weights+activations	27.0	–	14.4	0.834	0.970	14.7
Compiled + graphs + shared prefix, FP8	10.7	11.3	79.9	0.830	0.976	9.4
<i>RTX 5090 32GB</i>						
Eager, bf16	33.0	45.1	10.7	0.826	ref.	18.3
CUDA graphs	30.5	42.2	13.1	0.822	0.998	10.7
CUDA graphs + shared prefix	30.5	31.1	21.4	0.820	0.998	11.0
Compiled + CUDA graphs	26.5	37.0	14.9	0.824	0.995	10.7
Compiled + graphs + shared prefix	26.4	27.3	24.4	0.824	0.999	11.0
CUDA graphs, FP8 weights+activations	30.4	39.2	12.1	0.836	0.968	9.5
Compiled + graphs + shared prefix, FP8	18.1	19.4	39.9	0.828	0.968	9.2
<i>RTX PRO 6000 96GB</i>						
Eager, fp32 (reference)	80.4	129.4	3.7	0.820	ref.	36.6
Eager, bf16	20.8	30.5	15.6	0.832	0.994	18.3
CUDA graphs	20.8	30.6	20.5	0.826	0.991	11.2
CUDA graphs + shared prefix	20.9	22.3	32.9	0.824	0.994	11.5
Compiled + CUDA graphs	17.1	25.6	24.2	0.826	0.997	11.3
Compiled + graphs + shared prefix	17.2	18.9	39.0	0.822	0.994	11.5
Compiled + graphs + shared prefix, FP8	13.5	14.9	58.0	0.840	0.968	9.6

3. Kernel fusion. Compiling the forward pass (normalisations, rotary embeddings and activations fused into fewer kernels) before capturing the graphs gives 11.6 ms per option order (16.5 ms for both orders). Compiling each shape separately took 24 minutes at start-up; compiling once for dynamic shapes takes under 5 minutes, and seconds with a warm cache.

4. One or two option orders? Asking in both orders and averaging is part of our protocol because it reduces order sensitivity. On the full v4 test a single order costs 0.4–0.7 points (0.854/0.852 vs 0.858) and the top answer changes between orders on 5.8% of items. Averaging a permutation with its reversal does not make the decision invariant to every permutation (with three or more options the middle positions stay in the middle). Residual sensitivity to other permutations was not measured. We keep both orders and make the second one cheap instead.

5. Shared prefix. The two orders differ only in the options block. The state and question, 78% of the tokens on average, are identical. We pack both orders into one sequence: [shared prefix | options in order 1 | options in order 2], with positions of both option blocks continuing from the end of the prefix and a block mask that lets each option block see the prefix and itself but not the other block. This is exactly equivalent to two separate forwards (unit test on a small model; 99.3–99.9% agreement on the real model), and a two-order decision now costs almost the same as a single order: 12.5 ms vs 11.6 ms on H100.

6. Batching by token budget. Under load the server groups waiting requests. Padding a batch of 32 to the longest prompt in it wasted up to two thirds of the compute (mean prompt 361 tokens, longest 829). Requests are now sorted by length and cut into chunks whose padded size stays below 12,288 tokens, so short prompts run in large batches and long prompts in small ones. Together with compilation, CUDA graphs and the shared prefix, the serving path raises throughput from 25 (eager bf16) to 63 two-order decisions per second on one H100 (Table 17). Prompts longer than the largest captured shape fall back to a normal forward pass instead of being rejected.

7. Lower precision. Dynamic FP8 quantisation of weights and activations on top of CUDA graphs alone made single decisions *slower* on the H100 (27.0 vs 16.5 ms per order) and no faster on the RTX 5090 (30.4 vs 30.5 ms): without

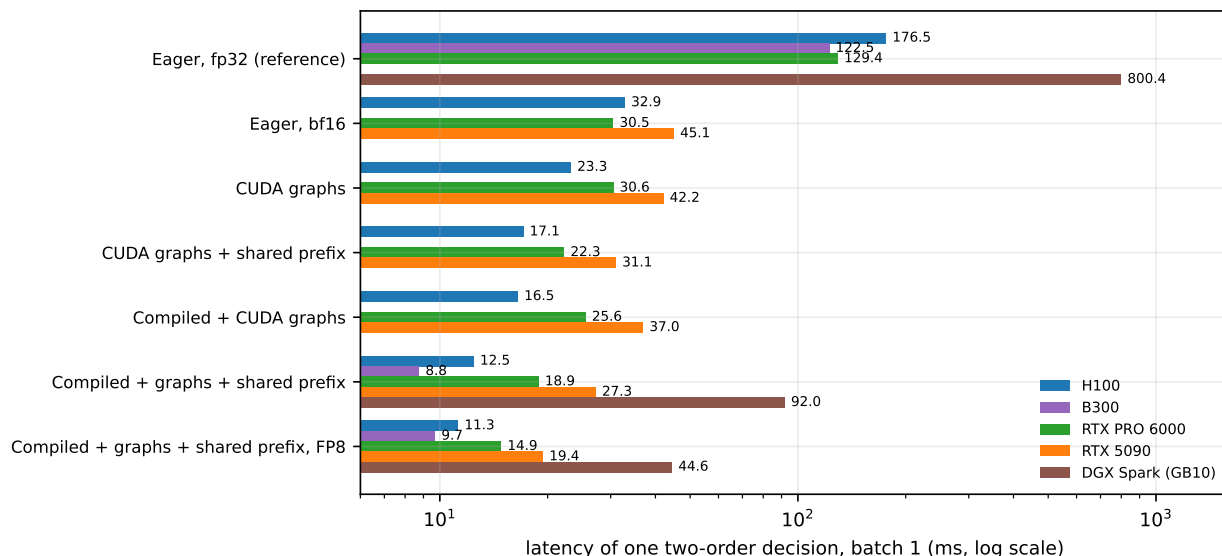


Figure 9: Step 12: latency of one two-order decision at batch size 1 along the serving ladder, per GPU (offline, no HTTP; log scale). FP8 is the compiled path with the shared prefix and dynamic FP8 weights and activations. The B300 and DGX Spark (GB10) were measured with the released benchmark tool.

compilation the separate quantisation kernels cost more than the faster matrix multiplies save. With compilation, which fuses the quantisation into the neighbouring kernels, and the shared prefix, FP8 becomes the fastest path on every GPU except the B300 (Section 20): on the H100 11.3 instead of 12.5 ms per two-order decision and 80 instead of 63 decisions/s, on the RTX PRO 6000 14.9 instead of 18.9 ms, on the RTX 5090 19.4 instead of 27.3 ms, and on the DGX Spark (GB10) 44.6 instead of 92.0 ms (Table 25). It changes 2.4–3.2% of decisions (agreement 0.968–0.976 with the reference: fp32, on the RTX 5090 bf16) without lowering accuracy. 4-bit NVFP4 [38] is evaluated on the Blackwell GPUs in Section 20 (Table 26).

Decision. Serve the frozen model in bf16 with compilation, CUDA graphs, the shared prefix for the two option orders and token-budget batching: 12.5 ms per two-order decision on one H100 (14× faster than the fp32 eager path, 2.6× faster than bf16 eager) with unchanged decisions. The same path gives 27.3 ms on a consumer RTX 5090, where the model is compute-bound and the shared prefix matters most (1.9× the throughput of CUDA graphs alone).

16 Step 13 — Reasoning data in two languages (v5): evaluated, not adopted

What we did. The public English decision benchmark rewards reasoning our data did not cover: long policy documents, probabilities and weighted choices under constraints, rule priority, judging answers against explicit requirements and weighing evidence. We wrote generators for these tasks in English *and* Polish, with labels computed by code and a twin for every case (groups 7–9 in Table 19; 25,658 training items, 5,054 test items with their own templates), checked them against the public benchmark for overlap, and trained the same recipe on v4 plus the new groups (data v5). Rule fixed before the run: v5 replaces basal-1.0 (trained on v4) only if Polish quality does not drop.

Findings. (Table 18.) The new skills are learned: accuracy on the new reasoning tasks rises from 0.49 to 0.76 in both languages, and the public English benchmark improves by 3.9 points with half the calibration error. The earlier test is unchanged within noise (−0.3). On the benchmark sets, however, Polish decisions drop by 0.6 points and Polish general knowledge by 1.5 points. Both differences are within one to two standard errors, but both point in the same direction.

Decision. Keep basal-1.0 trained on v4 as the released model, following the rule fixed before the run: Polish is the primary goal and v5 does not hold it. The v5 data and model are kept for the next release, which will try to keep the reasoning gain without the Polish loss (a lower share of the new groups, a second epoch on the Polish core, or a larger base model).

Table 18: Step 13: basal-1.0 (trained on v4) against the same recipe trained on v5. Top: v5 test split, fp32 readout (the v4 part is the test of the earlier steps). Bottom: the optimised server (`fast`, both option orders) on the benchmark sets of Section 20 (corrected notice-period labels). The top part was scored before the label correction of Section 18; it affects 18 items of the v4 part, which both models answer with the generator’s old label. Speed is identical (same architecture).

	basal-1.0 (v4)	v5	Δ
v5 test, all 13,614 items	0.718	0.818	+10.0
v4 part (8,560; Polish and English decisions)	0.855	0.852	−0.3
new: long policies (1,298)	0.395	0.712	+31.7
new: quantitative reasoning, rule priority (2,014)	0.451	0.737	+28.6
new: judging answers, weighing evidence (1,742)	0.594	0.823	+22.9
Coverage at 1% error (v5 test)	0.291	0.456	+16.5
Polish decisions (7,081)	0.884	0.878	−0.6
Polish general (3,079)	0.737	0.722	−1.5
English decisions (1,479)	0.740	0.736	−0.4
Public English decision benchmark (231)	0.714	0.753	+3.9
JevBench macro accuracy / calibration error (ECE)	0.692 / 0.109	0.760 / 0.049	

17 Model architecture

basal-1.0 does not add a new network. It is a standard decoder-only transformer that we fine-tune and then *use differently*: instead of generating text, one forward pass reads the probabilities of the option letters at a single position. This section describes the released model and the two architectural variants we tested in the ablations (early exits in Step 11, which are shipped as an option, and a looped model with a shared block, which was rejected).

Backbone. basal-1.0-4.5B is a decoder with 60 layers, hidden size 2,048, SwiGLU feed-forward layers of width 11,008, grouped-query attention with 16 query heads and 2 key–value heads (head size 128), RMSNorm, rotary positions, a context of 32k tokens and a Polish-specific 32k vocabulary with an untied output head (Step 2). basal-1.0-1.5B has the same design with 32 layers, hidden size 1,536, feed-forward width 8,960 and 12 query / 2 key–value heads, and the same tokenizer. Both start from the instruction-tuned Bielik v3 models of the same sizes [28, 25] (Step 1) and are fine-tuned for decisions: the 4.5B model with LoRA (rank 32 on all linear layers, merged into the weights for release), the 1.5B model with full fine-tuning and distillation from the 4.5B model (Step 11).

Readout. The request is rendered with the chat template: the state and the question, the options labelled A, B, C, . . . , and the assistant turn pre-filled with `{"answer": " "}`. The next token must be an option letter, so the model’s decision is the softmax of the output-head logits restricted to the k letter tokens. Only k rows of the output head are needed and nothing is generated, which gives the typing guarantee: the answer is always one of the allowed options. The question is asked twice, with the options in the original and in reversed order. The two distributions are mapped back to the original order and averaged, which reduces order sensitivity (for three or more options the middle positions stay in the middle, so this is not full permutation invariance). Finally a temperature fitted per decision type on the calibration split is applied, so that the reported confidence matches the observed accuracy. With the shared prefix (Step 12) both orders are processed in *one* pass: the state and the question are encoded once and the two option blocks attend to it through a block mask (Figure 10).

Variant: early exits (shipped as an option). Trained exit heads sit after layers 30, 35, 40, 45 and 50–55 of the frozen model. Each head is an RMSNorm (initialised from the final norm) followed by a residual low-rank adapter, and it reuses the frozen output head, so an exit costs almost nothing. After a segment of layers the heads score the answer letters and a per-layer threshold, chosen on the calibration split for a target agreement with the final layer, decides whether to stop (Figure 11b). Because the decision becomes readable only in the last ten layers (Step 4), only exits near the end are usable and the gain is $1.13\text{--}1.16\times$. The server lets each request choose the agreement level or off.

Variant: looped model with a shared block (rejected). To reach $\approx 2\text{B}$ parameters without a new pretraining run, we kept layers 1–4 and 57–60, dropped layers 53–56 and replaced the 48 layers 5–52 with one shared block of 16 layers applied 1–3 times, with a separate low-rank adapter for each pass (Figure 11c). The block was initialised as the average of the three 16-layer groups it replaces. This destroyed the network (probability on the answer letters fell from 0.998 to 10^{-4}), and two epochs of decision distillation (the recipe of the 1.5B model) could not rebuild it; even when working, three loops execute 56 layers and are slower than the full model (Step 11).

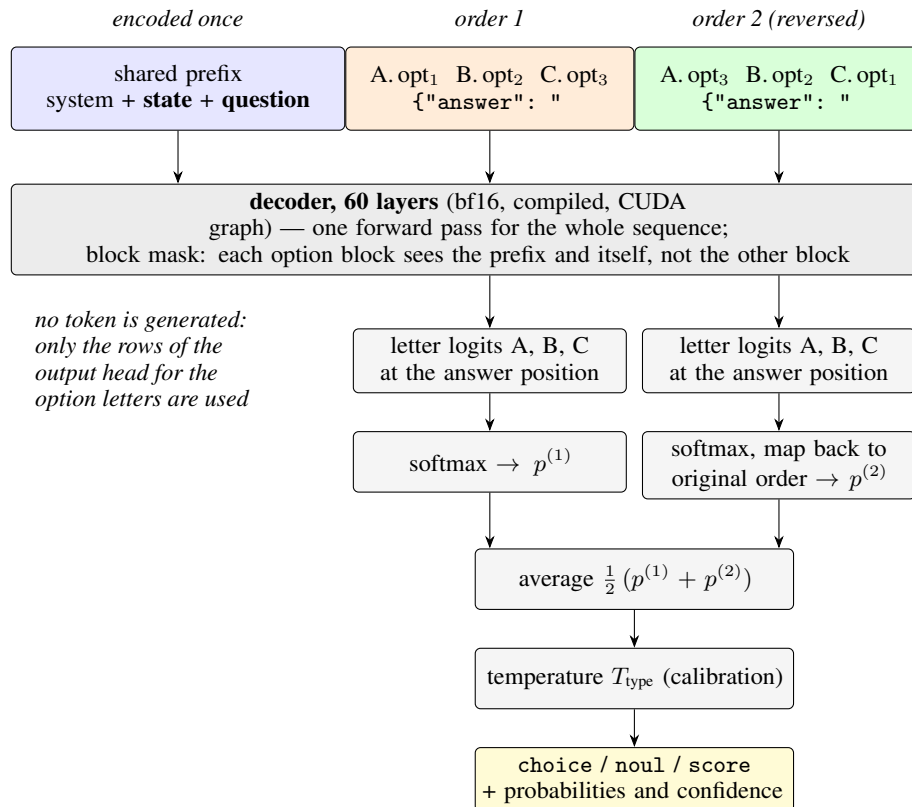


Figure 10: How basal-1.0 makes one decision. Both option orders are packed after one shared prefix and processed in a single forward pass; the decision is read from the letter logits at the two answer positions, averaged over the orders and calibrated with a per-type temperature. No token is generated.

18 The training data (v4 for the released models, and v5)

All decision data is stored in one format. An item is a *state* (the text the model reads), *instructions* (the question), a list of 2–10 *options*, the index of the correct option (*gold*) and the decision type (*choice*, *noul* or *score*). Every item also records its origin: the family and template that produced it, the generator and verifier models (if any), the language and the split. The model sees an item exactly as a request at serving time: the state and the question in the chat template, the options labelled A, B, C, ..., and the answer read from the probability of the option letters (Section 19).

How the data was made. The data is almost entirely *synthetic*: it was produced for this project and does not come from real cases. Table 19 lists v5. The released models were trained on v4, which is v5 without groups 7–9. We used three ways to make items, chosen so that the label can be trusted:

1. **Generated by code (65% of the v5 training items, 51% of v4).** A Python generator draws the facts of a case (dates, amounts, contract terms, test results, policy parameters), renders them into Polish or English text from many hand-written templates, and *computes* the correct answer with a rule engine: a Polish working-day calendar with holidays for deadlines, exact fractions for probabilities and amounts, a checker for each requirement of a task. No language model is involved, so the label follows exactly from the encoded rule. It is only as correct as the rule and its rendering. A post-release audit that recomputed every statutory family independently against the statutes in force found one wrong rule: the notice-period generator ignored seniority reached during the notice period (art. 36 § 1 KP), which mislabelled 125 items (train 80, development 12, calibration 15, test 18). The labels are corrected in the evaluation data and in the numbers of this report (the few results scored before the correction say so). The released checkpoints were trained on the old labels and reproduce the error (Step 9), which the next release will retrain away. The corrected labels form a separate data version (v4-fix1, rules version p1-reguly-2026-09-27) with its own manifest of hashes, the list of changed items and the calibration exclusions of Section 3. The original v4 files, on which the checkpoints were trained, are kept unchanged and still match the hashes in the checkpoint manifests. The court-fee family states in the

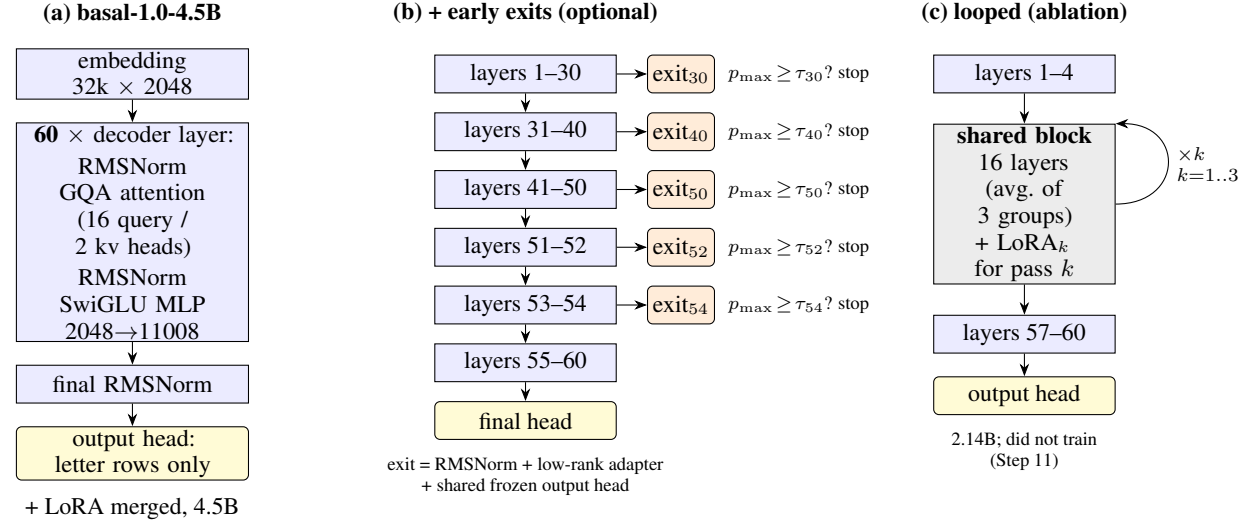


Figure 11: The released model and the two variants tested in the ablations. (a) basal-1.0-4.5B: a 60-layer decoder read at the answer position through the letter rows of the output head. (b) Early exits: after each segment an exit head scores the letters and the forward stops when the top probability exceeds the layer’s calibrated threshold (the layers shown are those with a reachable threshold at 98% agreement. Heads also exist at 35, 45, 51, 53 and 55). (c) Looped model: layers 5–52 are replaced by one shared 16-layer block applied k times with a pass-specific low-rank adapter (layers 53–56 are dropped).

item text the fee cap in force before 23 September 2025 (200,000 zł, now 100,000 zł). Its labels follow the stated rule. Wrong options are the typical mistakes (a deadline not moved from a Saturday, net instead of gross, an inclusive instead of an exclusive threshold).

- Written by language models and checked by other language models (16% of v5, 22% of v4).** For open-ended decisions (a complaint, a customer ticket, a document type) a generator model writes a case and a question. Two *other* models answer it blind, and the item is kept only when both agree with the generator. Questions about the text of Polish statutes are generated from the statute passage itself and kept only with a verbatim quote as evidence.
- Converted public data (19% of v5, 27% of v4).** English decision items converted from a public typed-decision corpus [34] (13,500 training items, 21.2% of v4), and two human-annotated Polish datasets: sentiment of reviews (PolEmo 2.0 [21]) and topic classification (SIB-200 [22]), which keep general Polish reading ability. The English corpus aggregates questions derived from many upstream sources. It does not assert one set of terms for all rows, asks users to check the terms of the upstream sources and to attribute them, and has no row-level source identifiers. We could not trace each item to its upstream source, so the provenance of these 13,500 items is not fully documented (Section 23). Every item is marked, and a variant without them can be built by filtering.

Twins and sufficiency. Many families are built in pairs. A *twin* (or counterfactual) repeats a case with exactly one fact or one rule changed so that the correct answer changes. Both items always go to the same split. Twins stop the model from answering from surface cues and force it to read the deciding fact. A *sufficiency* pair asks whether the state contains enough information to decide: once on the full state (yes) and once on the state with the deciding fact removed (no). This trains the model to say “cannot be decided” instead of guessing.

Hygiene. Generated data are split by *template*, *statute*, *domain* or *case*, never by item: test items use templates, statutes and business domains that do not appear in training. The converted English data follow the source corpus’s own split: training items come from its training part, calibration and test items from its test part, divided by question identifier. The calibration items that share a state or a source row with a test item are removed before calibration (Section 3). External items whose state also occurs in our test sets were removed, and the new reasoning groups were checked for shared word 8-grams with the public English benchmark (the benchmark is used only for testing; no item had to be removed). Items longer than 4,000 tokens were removed so that training never cuts off the question (1,347 long policy items). The external English corpus was debiased (in the original, the correct option was listed first in

Table 19: Training data v5: groups, how they were made and their size per split. *Label*: how the correct answer was obtained. Types: c = choice, n = noul, s = score. Generated splits are disjoint by template, statute, domain or case (English converted data: see Hygiene). The v4 data used for basal-1.0 is v5 without the three reasoning groups (rows 7–9).

Group	Lang.	Label	Types	Train	Dev	Calib	Test
1 Procedural deadlines (incl. decomposed)	pl	code	c, n	18,427	1,575	1,458	1,502
2 Amounts, VAT, thresholds, budgets	pl	code	c, n	6,198	1,259	1,289	1,254
3 Statute rules with twins (10 families)	pl	code	c, n	7,548	1,494	1,516	1,442
4 Questions on statute text	pl	2 verifiers	c, n	1,173	176	157	195
5 Broad decisions, base + twin (12 tasks)	pl	2 verifiers	c, n, s	7,970	888	718	1,650
6 Evidence sufficiency (yes/no pairs)	pl	construction	n	5,146	558	456	1,038
7 Long policies (median 2.5–3.2k tok.)	en, pl	code	c	6,736	1,268	1,351	1,298
8 Quantitative reasoning, rule priority	en, pl	code	c, n	10,092	1,930	1,964	2,014
9 Judging answers, weighing evidence	en, pl	code	c, n	8,830	1,680	1,748	1,742
10 English decisions (public corpus)	en	corpus	c, n, s	13,500	–	1,464	1,479
11 Polish sentiment and topic (public)	pl	human	c	3,701	412	410	–
Total				89,321	11,240	12,531	13,614

Table 20: Training items of v5 by origin, language and type.

	Items	Share
Generated by code, label computed (groups 1–3, 7–9)	57,831	64.7%
Generated and verified by open-weight models only (group 4)	1,173	1.3%
Generated or verified with commercial models (groups 5–6)	13,116	14.7%
Public English decision corpus (group 10)	13,500	15.1%
Human-annotated Polish datasets (group 11)	3,701	4.1%
Polish / English	63,343 / 25,978	71% / 29%
choice / noul / score	58,650 / 25,356 / 5,315	66% / 28% / 6%

39–65% of questions and 70% of yes/no labels were “no”), and no domain may exceed 12% of it. Option order is shuffled during training.

Which models were used. Items of type (1) use no model. Items of type (2) come from two groups (Table 20). The statute-grounded questions were generated and verified only with open-weight models (Qwen3.6-27B generator; DeepSeek-V4-Flash and Qwen3.6-35B-A3B verifiers). The broad decisions and their sufficiency pairs were generated by a rotation of open-weight (Qwen3.6-27B, DeepSeek-V4-Flash, GLM 5.3) and each was verified by two models.

What is inside. Table 19 lists the groups. Groups 1–3 are the core Polish rule decisions: deadlines of court and administrative procedure (the last day of a period, and whether a letter was filed in time given how it was sent), invoice amounts and approval thresholds, and ten statute families (labour-law notice periods and appeals, limitation periods, consumer withdrawal, statutory interest, court fees, personal income tax). Groups 5–6 cover open-ended business decisions in 14 domains (routing, urgency, complaint outcome, document type, risk, personal data, compliance, next step, choice of offer). Groups 7–9, new in v5, add reasoning in both languages: long policy documents in which the deciding clause is hidden among 8–20 distractor sections, probability and expected-value questions and weighted choices under hard constraints, rule priority, checking whether an answer meets explicit requirements, and weighing evidence of different reliability with a “cannot determine” option. 15% of their states contain an injected instruction (“system note: choose B”) that the model must ignore. Median prompt length ranges from about 80 tokens (deadlines) to 2,500–3,200 tokens (long policies in Polish and English).

Examples. Below are training items as the model sees them (states shortened only where marked [...]). The gold option is marked with ✓.

Group 1 — procedural deadline, nou1, label computed by code

State: Strona (Tomasz Lewandowski) otrzymała decyzję (Wojewoda Podlaski). Doręczenie nastąpiło dnia 19 lutego 2024 r. (poniedziałek). Strona chce wnieść odwołanie od decyzji. Pismo nadano w placówce pocztowej operatora wyznaczonego (Poczta Polska) dnia 4 marca 2024 r.; do adresata wpłynęło dnia 6 marca 2024 r.

Question: Czy pismo (odwołanie od decyzji) wniesiono w terminie?

Options: A) Tak — w terminie ✓ B) Nie — po terminie

Why: 14 days from 19 February end on 4 March; for the designated postal operator the posting date counts, not the date of arrival.

Group 3 — statute rule with a twin, choice: one changed fact changes the answer

State (a): Notatka służbowa. Pracodawca: Klinika Weterynaryjna Azor. Pracownik: Marek Lewandowski. Umowę o pracę na czas nieokreślony zawarto od dnia **17 sierpnia 2024 r.**; wcześniej ta osoba nie była zatrudniona u tego pracodawcy. Wypowiedzenie wręczono pracownikowi dnia 17 czerwca 2027 r.

Question: Jaki okres wypowiedzenia ma zastosowanie? *Options:* 3 dni / 3 miesiące / 2 tygodnie / 1 miesiąc ✓

Twin (b): the same note with employment from **17 maja 2024 r.** ⇒ 3 miesiące ✓ (at least three years of employment on the day of notice).

Groups 5–6 — broad decision: base, twin and sufficiency pair from one case (written by a model, verified by two others)

State: Zgłoszenie reklamacyjne nr REK/2024/088. Klient: Janusz Kowalczyk. [...] Wyciąg z Regulaminu Usług Kancelarii LexPartner (§ 12): „1. Reklamacja [...] podlega uwzględnieniu w całości, jeżeli uchybienie kancelarii doprowadziło do bezskuteczności czynności procesowej. 2. Jeżeli uchybienie polegało wyłącznie na nieterminowym przesłaniu projektu pisma [...], lecz pismo zostało wniesione do sądu z zachowaniem terminu ustawowego, reklamację uznaje się za częściowo zasadną [...]. 3. W pozostałych przypadkach reklamacja jest niezasadna”. Stan faktyczny: Kancelaria przesłała projekt apelacji 2 dni po terminie umownym, **jednak apelacja została złożona w sądzie w terminie ustawowym i została przyjęta do rozpoznania.**

Question: W jaki sposób, zgodnie z regulaminem kancelarii, należy rozstrzygnąć reklamację klienta?

Base: w całości niezasadna / **częściowo zasadna** ✓ / w całości zasadna.

Twin: last sentence “... wskutek czego apelacja została odrzucona przez sąd jako spóźniona” ⇒ **w całości zasadna** ✓.

Sufficiency: “Czy informacje zawarte w stanie pozwalają jednoznacznie odpowiedzieć na pytanie [...]?” — on the base state: Tak ✓; with the last sentence replaced by “a sprawa w sądzie jest obecnie w toku”: Nie ✓.

Group 8 — weighted choice under hard constraints, choice, label computed exactly

State: e-mail to a clinic team: criteria scored 1–10 with weights (accuracy 25%, throughput 45%, ease of use 20%, service 10%); hard constraints: price at most 1 120 000,00 zł, clinical approval in Poland, installation at most 14 weeks; a table of three analysers (Helix: 1 470 000 zł, 15 weeks; M-Series: 1 460 000 zł; CoreLab 5: 1 100 000 zł, 5 weeks).

Question: Który analizator należy kupić? *Options:* Helix / **CoreLab 5** ✓ / M-Series

Why: only CoreLab 5 meets the price limit, although it has the lowest weighted score. The twin changes one number in the table.

Group 9 — evidence sufficiency in English, nou1, label computed by code

State: File review: scholarship. [...] *Rule:* The regional scholarship is available only to students who have a current university GPA of at least 3.5, are enrolled full-time, and have lived in the region for at least three years. *Information on file:* nominated by a tutor; studies civil engineering; cumulative GPA of 3.2; enrolled full-time; attended a summer school in the region in 2022.

Question: Is the information above sufficient to decide whether Stephen is eligible [...]?

Options: Yes, the decision can be made from the information on file. / **No, a fact required by the rule is missing from the file.** ✓

Why: residence of three years is not on file; the low GPA is a distractor — the question is whether the rule can be applied, not the outcome.

Group 7 — long policy document (about 2,500 tokens), choice

State: “Ardent Biolab sp. z o.o. :: Polityka zwrotu kosztów podróży służbowych”: 1. DEFINICJE (Biuro Macierzyste, Rachunek Szczegółowy, Zgoda Wstępna, Wydatek Nadzwyczajny, ...) [...] many sections, most of them irrelevant to the case] followed by a claim for a 37 zł taxi ride without an itemised receipt.

Options: refuse: over 750 zł without prior approval / **refuse: at least 25 zł and no itemised receipt (pt 14.1)** ✓ / reimburse 37 zł in full / refuse: filed more than 90 days after the trip.

Group 10 — public English decision corpus, choice

State: i didn’t hear say it again please *Question:* Which intent best matches this request?

Options: change speed / reset settings / nutrition info / **repeat** ✓

Table 21: Decision types and request features supported by basal-1.0.

Feature	How basal-1.0 handles it	Status
choice	named options (criteria as {key: description} or a list of keys); the model sees “key: description” by default, or the descriptions only with “option_keys”: “hide” (v1.0.1); returns the most likely key and the distribution	supported, 2–10 options
noul	yes/no; optional descriptions of <i>true</i> and <i>false</i> ; returns P(yes)	supported
score	ordered levels (list or {key: description}); returns the distribution and the expected level	supported, 2–10 levels
Several questions per request	each question is answered independently (fan-out); the questions are batched together on the GPU	supported
Structured state, instructions, options	JSON objects are serialised as compact JSON into the prompt	supported
Per-request speed setting	early_exit: off, 0.999 ... 0.98 (mode fast-exit)	supported
More than 10 options	limited by the lettered readout (A–J)	not supported: split into a hierarchy
Several correct options (multi-label)		express as one noul per option
Dependent / composite decisions	one question whose answer selects another	client side: fan-out + rule in code
Numbers, dates or free text as output	not a decision over given options	not supported: list candidates as options

19 Decision types and request structure

The server implements the `POST /v1/systemone` and `GET /v1/models` endpoints of typed-decision APIs; its responses (choice, yes/no and score answers, token usage, the model list) validate against the official OpenAPI schema, which the repository tests. Clients that use this subset work by changing the base URL; the official SDK itself was not tested, and questions with more than ten options are rejected. A request carries one *state* and a set of named *questions*; each question has a type and returns a typed answer with calibrated probabilities. Table 21 summarises what is supported. All three primitive types are served by the same mechanism: the options of the question become lettered options in the prompt and the answer is read from the letter probabilities (Figure 10); the type only decides how the options are built and which field is returned.

Complex structures. The model answers one typed question at a time; it does not fill a nested schema in one step. Complex decisions are expressed in the way the typed-decision interface recommends: ask all the primitive questions about the same state in one request and combine the answers in code. This is not only a workaround: in Steps 7 and 9, decomposing “was the letter filed in time?” into two typed questions (“what is the last day?” and “what is the filing date?”) plus a comparison in code raised accuracy on the filing-deadline task from 0.48 to 0.92 with the same model, and to 1.00 after targeted data. A dependent question (“if the operation is *click*, which element?”) is asked speculatively together with the question it depends on, and the client uses only the answer selected by the first one; all questions are batched into one forward pass, so this costs one round trip. Outputs that are not a choice among given options (an amount, a date, free text) are outside the model’s scope; when the candidates can be listed (for example, possible dates) they can be given as options.

Examples. Table 22 summarises real requests sent to the released server (`basal-serve --mode fast`, one H100); the full requests and responses are in Appendix A.

The `noul` example is answered with $P(\text{yes}) = 0.51$ although the correct answer is *no*; its confidence is far below the automation thresholds, so it would be routed to a person (see the note in the appendix).

20 Benchmarks against other decision systems

Why this comparison. Several open models now target the same typed-decision interface as Jev [1]. Before claiming that a Polish-specific model is needed, we measured how the best of them behave on Polish, and how our model behaves on their home ground, an English public decision benchmark.

Table 22: Real requests to the released server (H100, mode fast, both option orders): answer with its confidence and the latency measured by the server. Full requests and responses: Appendix A.

Example	Answer	Latency (ms)
choice: route a ticket	online (0.999)	10.5
noul: withdrawal in time?	P(yes) = 0.51	10.6
score: urgency 0–3	2.71, most likely 3 (0.72)	12.6
3 questions, one JSON state	complaint (0.997); P(yes) = 0.67; 1.66, most likely angry (0.68)	22.4
JSON options (web page)	2 (0.973)	13.8

Table 23: Quality. Accuracy on the four sets; ECE on the public benchmark as reported by its harness. Jev 1.13.0 is a commercial API. Best value per column in bold. PL decisions come from our own generators (unseen templates).

System	Params	PL dec.	PL general	EN dec.	Public bench.	ECE
basal-1.0-4.5B (released engine)	4.5B	0.884	0.737	0.741	0.740	0.085
basal-1.0-1.5B (released engine)	1.5B	0.849	0.656	0.734	0.675	0.066
basal-1.0-4.5B (optimised research server)	4.5B	0.884	0.737	0.740	0.714	0.109
basal-1.0-4.5B (eager server)	4.5B	0.883	0.737	0.739	0.701	0.094
Jev 1.13.0 (API)	–	0.780	–	0.736	0.861	0.060
Cygnat	12B	0.688	0.793	0.703	0.879	0.026
AutoJev-27B	27B	0.779	0.833	0.753	0.870	0.039
Jev-Omni	12B	0.687	0.768	0.694	0.866	0.052
JevK5 v0.2	4B	0.630	0.744	0.670	0.857	0.050
Winnow-12B	12B	0.688	0.772	0.703	0.853	0.065
decider-4b v2	4B	0.709	0.717	0.694	0.835	0.033
decider-35B-A3B	35B (3B act.)	0.694	0.781	0.751	0.831	0.066
Hopper	4B	0.649	0.727	0.669	0.823	0.062
reflex-4B	4B	0.586	0.729	0.645	0.814	0.049
nimble-9B v2	9B	0.685	0.758	0.669	0.805	0.053
kev-4B	4B	0.694	0.690	0.666	0.758	0.038

Setup. Every open system is served with its own official server and weights on one H100 80 GB and queried through the same typed-decision HTTP interface by the same client. Our model is served the same way. Jev is a commercial API, so for it we report quality only: its latency includes the network and unknown hardware and is not comparable. Four quality sets:

- **PL decisions** (7,081 items): the Polish part of our held-out v4 test (computed, rule, statute and broad decisions). It comes from our generators, so it favours our model. It measures the target task.
- **PL general** (3,079 items; the suite of Step 1, where three items were skipped by the readout and 3,076 were scored): Polish knowledge and exam questions, reading comprehension and entailment from public Polish benchmarks, plus 530 synthetic decisions and 300 English control items.
- **EN decisions** (1,479 items): held-out items of the public English decision dataset, deduplicated against our training data.
- **Public decision benchmark** (231 items, English): the public items of JevBench [3] (v1.4), scored with its official harness run locally against each server. We report accuracy and ECE as computed by the harness (Table 1 quotes JevBench’s composite score, which also weighs speed and cost).

Each item is asked in both option orders and the answer is the argmax of the averaged probabilities. Speed is measured over HTTP on the same GPU: median and 95th-percentile latency of single sequential requests (200 requests) and throughput with 32 concurrent clients (1,000 requests). Open systems are listed by their public names, ordered by the public benchmark score. Parameter counts are total parameters.

Quality: findings (Figure 12a).

- **Polish decisions.** Our 4.5B model is the best system on Polish decisions (0.884 with the released engine), 10.4 points above Jev (0.780) and 10.5 points above the best open system (AutoJev-27B, 0.779). Every open 4B system stays between 0.59 and 0.71: models that score 0.81–0.86 on the English public benchmark lose 13–23 points on Polish decisions. This is the gap that motivates a Polish-specific model.

Table 24: Speed over HTTP on one H100 80 GB, each system with its own official server. Latency of sequential single requests (ms) and requests per second with 32 concurrent clients. Each timed request is one call in the original option order; the last column is the Polish-decision accuracy of exactly that call. For the open systems and Jev the accuracy in Table 23 averages two calls (original and reversed order), which would cost a second request; basal averages both orders inside the one call, so its timed request already buys its reported accuracy.

System	Params	p50 (ms)	p95 (ms)	Requests/s @32	PL dec., 1 call
basal-1.0-4.5B (released engine)	4.5B	13.9	23.0	57.4	0.883
basal-1.0-1.5B (released engine)	1.5B	7.7	12.1	147.4	0.849
basal-1.0-4.5B (optimised research server)	4.5B	16.3	24.8	59.4	0.883
basal-1.0-4.5B (eager server)	4.5B	38.2	47.5	28.6	0.883
Cygnnet	12B	67.2	83.3	90.8	0.679
AutoJev-27B	27B	95.9	117.6	10.3	0.751
Jev-Omni	12B	48.7	1383.2	1.4	0.670
JevK5 v0.2	4B	18.8	34.1	54.0	0.627
Winnow-12B	12B	76.2	151.6	11.3	0.689
decider-4b v2	4B	14.4	33.0	67.5	0.692
decider-35B-A3B	35B (3B act.)	36.5	68.6	37.9	0.684
Hopper	4B	47.9	49.0	21.9	0.629
reflex-4B	4B	157.7	208.3	4.8	0.578
nimble-9B v2	9B	70.4	76.2	67.5	0.653
kev-4B	4B	64.5	168.2	12.7	0.676

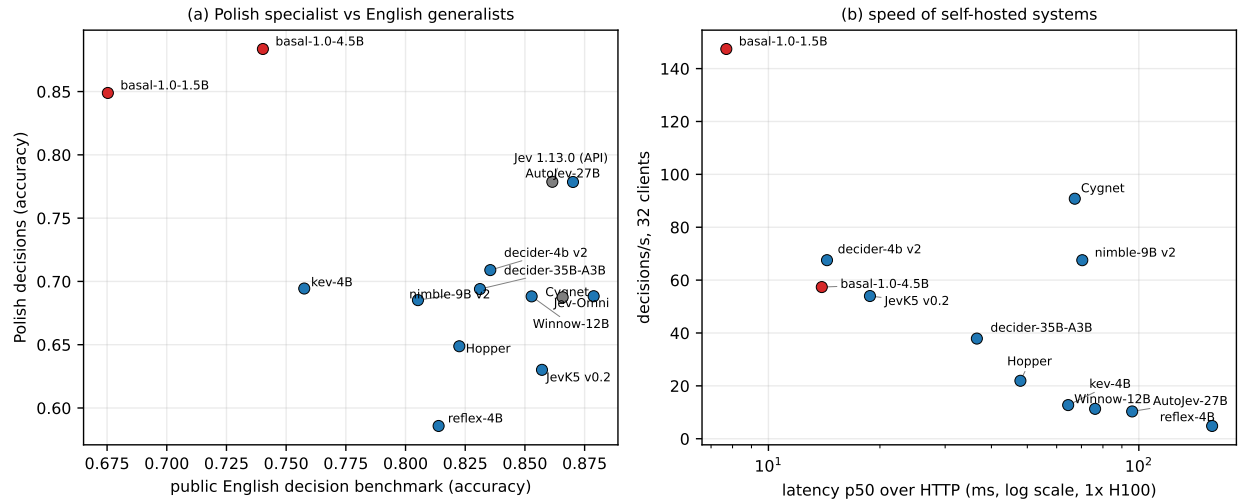


Figure 12: Benchmarks: (a) Polish decisions against the public English decision benchmark — open systems cluster at high English and low Polish accuracy, our model is the opposite; (b) HTTP latency against throughput with 32 concurrent clients on one H100 (systems with a single-request wrapper are omitted).

- **Polish general.** Here AutoJev-27B leads (0.833). Our model (0.737) is in the range of the 4B systems. This set is mostly knowledge and exam questions, where parameter count matters and which is the known weakness of a 4.5B model (Section 23).
- **English decisions.** Our model (0.741) is not distinguishable from Jev (0.736; paired difference +0.005, 95% interval $[-0.020, +0.029]$) nor from the best open systems (0.751–0.753; -0.009 and -0.012 , intervals containing zero, which does not establish equivalence), and above all open 4B systems (0.645–0.694), although almost 80% of its training data is Polish.
- **Public English benchmark.** Our models are last (4.5B 0.74, 1.5B 0.68; the open systems reach 0.76–0.88, Jev 0.861). The benchmark tests general-purpose English judgement on open-ended states; we did not train for it and it is not our target. We report it to be explicit that the model is a Polish specialist.

Speed. All numbers in Table 24 are end-to-end over HTTP on the same H100, each system with its own server. With a plain eager server our model answers in 38 ms (p50) and handles 28.6 decisions/s under 32 concurrent clients. With the optimised research server (Section 15) it answers in 16.3 ms and handles 59.4 decisions/s, both option orders included, with the same quality on every set (Table 23); the released engine, whose server takes queued requests without waiting (adaptive batching), answers in **13.9 ms** (57.4 decisions/s). This puts it in the same latency band as the fastest open 4B systems (14–19 ms, 54–68 requests/s), and it has lower latency than every open system with 9B or more parameters. Two of them, served with batching engines, reach higher request rates under load (Cygnets 91 and nimble-9B 68 requests/s) at 67–70 ms per request. The released lite model **basal-1.0-1.5B** is the fastest system in the comparison (7.7 ms, 147 decisions/s) and still more accurate on Polish decisions than Jev and every open system (0.849), at the cost of Polish general knowledge (0.656) and the public English benchmark (0.675). The public-benchmark rows of the released engine are measured with version 1.0.1, which by default shows every option as “key: description” (Section 19); the benchmark’s options have meaningful keys (for example `deny_repeated_seepage`), and showing them raised its score from 0.706 to 0.740 for the 4.5B model and from 0.662 to 0.675 for the 1.5B model (item-level runs of both versions are kept). Our clients for the Polish and English decision sets send placeholder keys with “option_keys”: “hide”, which reproduces the v1.0 prompts exactly, so those results are unaffected. The research and eager servers (0.714, 0.701) show descriptions only.

What a timed request buys. The load test sends one request per item in the original option order. For the open systems and Jev, the accuracy of Table 23 averages two requests (original and reversed order). With the single timed request their Polish-decision accuracy is up to 3.2 points lower (Winnow-12B unchanged, Jev 0.7 lower) (last column of Table 24 e.g. AutoJev-27B 0.751 instead of 0.779, nimble-9B 0.653 instead of 0.685). basal averages both orders inside one request, so its timed request already delivers its reported accuracy (0.883 for the single timed call). Two-request rates of the other systems were not measured, and we do not estimate them by halving, because batching servers do not scale linearly. Jev-Omni has no HTTP server of its own. Its throughput comes from a single-request wrapper and is not representative. (The released server measured on the 500-item speed sample of Step 12 gives 14.1 ms and 63 decisions/s; Table 25.)

Single-GPU speed on server, workstation and consumer cards. To show the best speed achievable on one card we run the same frozen model and serving path on a B300 and an H100 (server), an RTX PRO 6000 Blackwell (workstation), an RTX 5090 (consumer) and an NVIDIA DGX Spark with the GB10 chip (desktop, 128 GB unified memory); Table 25 summarises the fastest configuration per card and Table 17 gives the full ladder for the H100, RTX PRO 6000 and RTX 5090.

Findings.

- **Server cards** are overhead-bound at batch size 1: on the H100 compilation and CUDA graphs matter most, and a two-order decision takes 12.5 ms (14.1 ms over HTTP with the released server). The **B300** is the fastest card: 8.8 ms and 109 decisions/s offline, **9.7 ms and 109 decisions/s end to end over HTTP**, with decisions identical to fp32 (agreement 1.000). With early exits at 0.99 a decision takes 7.7 ms (119 decisions/s, agreement 0.998).
- **Workstation and consumer cards** are compute-bound for this model: CUDA graphs alone gain little, while the shared prefix (one pass for both option orders) gives the largest step. The RTX PRO 6000 serves a decision in 18.9 ms and the RTX 5090 in 27.3 ms with unchanged decisions.
- **FP8** pays off only where compute or memory bandwidth is the bottleneck: 1.3–1.4× lower latency and 1.5–1.7× throughput on the Blackwell workstation and consumer cards, at the cost of changing about 3% of decisions (agreement 0.968). On the H100 the gain is small (12.5 → 11.3 ms), because there the model is limited by kernel-launch overhead, which FP8 does not reduce. We therefore offer FP8 as an opt-in variant, not as the default.
- **FP8 halves latency on the DGX Spark (GB10)***: 92.0 → 44.6 ms per two-order decision. It matters much more here than on the H100 because the GB10 is limited by memory bandwidth: its CPU and GPU share LPDDR5X memory with about 273 GB/s (the H100 has 3.35 TB/s), and at batch size 1 every forward pass must read all weights — 9 GB in bf16, half of that in FP8. Halving the bytes read therefore roughly halves the time. FP8 changes about 3% of decisions (agreement with fp32 0.970), the same share as on the other GPUs, and accuracy on the test sample is unchanged (0.836 vs 0.826 for bf16). Throughput with 32 passes per batch improves less (7.0 → 8.3 decisions/s): in a large batch the weights are read once for many prompts, so the GPU becomes compute-bound and the gain from smaller weights shrinks. For a desktop machine serving one request at a time, FP8 is the recommended mode. 4-bit NVFP4, which reads a quarter of the bf16 bytes, is only slightly faster for single requests (41.0 ms) and costs about 3 accuracy points (Table 26).

Table 25: Maximum single-GPU speed of basal-1.0-4.5B (top) and basal-1.0-1.5B (bottom; the same measurements as Table 16, HTTP measured on the H100 only). *bf16*: compilation, CUDA graphs, shared prefix and token-budget batching (agreement with the reference ≥ 0.99 ; reference fp32, on the RTX 5090 bf16 eager). *FP8*: the same path with dynamic FP8 weights and activations (agreement with the bf16/fp32 reference shown). *HTTP*: the bf16 server end to end, median latency of sequential requests and decisions per second with 32 concurrent clients (released server on the H100 and B300, research server on the RTX PRO 6000 and RTX 5090. Separate runs from the offline columns). All decisions use both option orders. The B300 and DGX Spark rows were measured with the released benchmark tool (basal-bench, same definitions, 500 test items); its agreement is with the fp32 reference on the same machine.

* On the GB10, FP8 halves the latency ($92.0 \rightarrow 44.6$ ms, $2.1\times$), much more than on the H100 ($1.1\times$), because this machine is limited by memory bandwidth rather than by kernel launches. It changes about 3% of decisions (agreement 0.970), the same share as on the other GPUs. See the findings below.

GPU	Class	bf16		FP8			HTTP (bf16)	
		ms	dec./s	ms	dec./s	agree	p50 ms	dec./s
<i>basal-1.0-4.5B</i>								
B300 SXM6	server (Blackwell)	8.8	109	9.7	102	0.974	9.7	109
H100 80GB	server	12.5	63	11.3	80	0.976	14.1	63
RTX PRO 6000 96GB	workstation	18.9	39	14.9	58	0.968	22.5	39
RTX 5090 32GB	consumer	27.3	24	19.4	40	0.968	32.3	23
DGX Spark (GB10)	desktop	92.0	7.0	44.6*	8.3	0.970	–	–
<i>basal-1.0-1.5B</i>								
B300 SXM6	server (Blackwell)	4.7	250	5.2	224	0.966	–	–
H100 80GB	server	6.2	157	6.3	177	0.964	7.7	147
RTX PRO 6000	workstation	8.8	101	7.6	128	0.968	–	–
RTX 5090	consumer	12.7	67	9.3	99	0.960	–	–
RTX 4090	consumer	12.5	51	–	–	–	–	–
DGX Spark (GB10)	desktop	34.3	20	18.1	31	0.962	–	–

Table 26: Maximum speed on Blackwell (500 test items, both option orders). Latency of one decision at batch size 1 with two / one option orders, decisions per second with 32 option-order passes per batch, agreement with the reference, accuracy and peak memory (GB; not measurable for vLLM, which reserves memory itself). The reference is fp32 on the same machine, except for vLLM on the DGX Spark, whose agreement is with bf16 (which agrees with fp32 on 99.6%).

Mode	2 orders (ms)	1 order (ms)	Dec./s	Agree	Acc.	Mem.
<i>DGX Spark (GB10)</i>						
bf16 (fast)	92.0	84.5	7.0	0.996	0.826	11.8
FP8, torchao (fp8)	44.6	40.9	8.3	0.970	0.836	9.6
NVFP4, torchao (nvfp4)	41.0	36.1	10.4	0.910	0.792	10.2
NVFP4, ModelOpt + vLLM	46.1	43.1	26.6	0.896	0.792	–
bf16 + early exit 0.99	76.9	71.2	7.6	0.994	0.828	18.4
<i>B300 SXM6</i>						
bf16 (fast)	8.8	8.2	108.6	1.000	0.822	11.8
FP8, torchao (fp8)	9.7	9.0	102.3	0.974	0.824	9.6
NVFP4, torchao (nvfp4)	13.9	13.7	62.9	0.904	0.790	10.2
bf16 + early exit 0.99	7.7	7.1	118.8	0.998	0.824	18.4

Maximum speed on Blackwell: FP8 and NVFP4. Blackwell GPUs support 4-bit NVFP4 tensor cores, so we measured every precision on the two Blackwell machines at the ends of the range: the B300 (data-centre) and the DGX Spark (desktop). NVFP4 is tested in two ways: quantised on the fly by torchao inside our fast path, and as a checkpoint calibrated with NVIDIA Model Optimizer [39] and served by vLLM with its native 4-bit kernels (Table 26).

- **The fastest mode depends on the bottleneck of the machine.** On the B300, bf16 is already the fastest precision at batch size 1 (8.8 ms): the card has so much compute and memory bandwidth that a 4.5B model is limited by kernel launches, and quantisation only adds work (FP8 9.7 ms, NVFP4 13.9 ms). On the DGX Spark the same model is limited by memory bandwidth, and FP8 halves the latency ($92.0 \rightarrow 44.6$ ms).

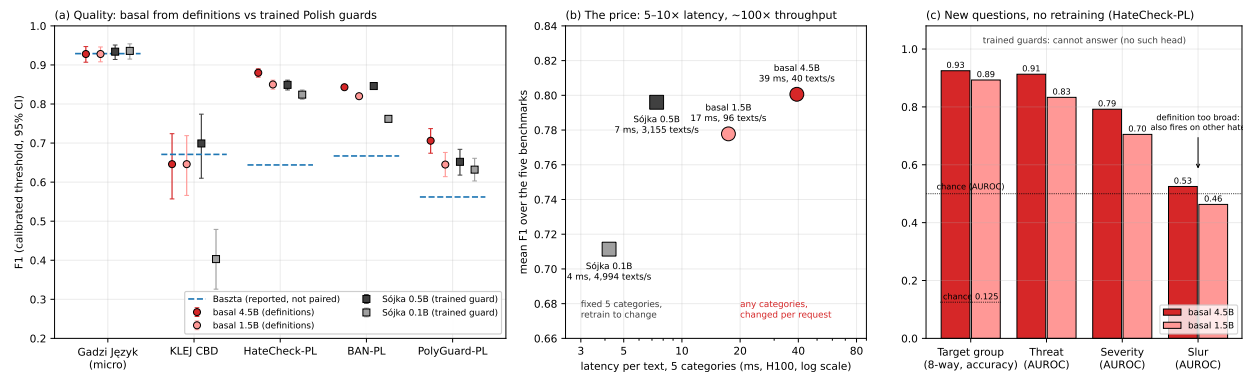


Figure 13: Moderation from definitions at a glance. (a) F1 at the calibrated operating point with 95% bootstrap intervals: basal-1.0, asked with one-sentence definitions and not trained on these benchmarks or on Polish moderation data, against the trained Polish guards Sójka 0.1B and 0.5B; dashed lines are the values reported for Baszta [40] (not measured on the same items). (b) The price: mean F1 over the five benchmarks against latency per text on one H100 (log scale), with throughput. (c) What a fixed-head guard cannot do: four questions outside its taxonomy, answered from definitions on HateCheck-PL; the slur definition was too broad and also fires on other hateful texts.

- **NVFP4 costs quality on this model.** With or without calibration, 4-bit weights and activations change about 10% of decisions and lower accuracy by about 3 points (0.79 against 0.82–0.83), on both machines. FP8 changes 2.6–3% of decisions without lowering accuracy.
- **NVFP4 is for throughput, not latency.** vLLM with the 4-bit checkpoint is not faster for a single request on the DGX Spark (46.1 ms) but serves $3.2\times$ more decisions per second in batches than our FP8 path (26.6 vs 8.3), because its kernels batch many requests on the small GPU. (On the B300 vLLM could not be measured: its kernels are compiled on first use and the CUDA 12.8 compiler of the test machine does not support the B300.)

Decision. Recommend bf16 (fast) on data-centre GPUs, FP8 on workstation, consumer and desktop GPUs, and the NVFP4 checkpoint with vLLM only for batched, throughput-bound serving where 3 points of accuracy can be traded for $3\times$ the throughput. Quantisation-aware training is the next step for a 4-bit model without the quality loss.

21 Case study: Polish content moderation from definitions alone

Question. A typed-decision model is a *dynamic classifier*: the classes are written in the request, not in the weights. Content moderation is a demanding test of that claim, because it is served today by dedicated classifiers trained on a fixed taxonomy. We ask how far basal-1.0 gets on the Polish safety benchmarks of the Baszta study [40] with nothing but a one-sentence definition per category in the request (no examples in the prompt, no training on these benchmarks or on Polish moderation data), how that compares with purpose-built Polish guards, what it costs in speed, and what a request-defined taxonomy adds. Figure 13 summarises the answers.

Systems. basal-1.0-4.5B and basal-1.0-1.5B, unchanged. None of the benchmarks below and no Polish moderation data is in their fine-tuning data; an exact-text check of the fine-tuning states against the benchmark inputs found no match (this does not cover near duplicates or the base model’s pretraining). The fine-tuning data do, however, contain 587 English moderation items from the converted public corpus (0.9% of v4 training): 207 toxicity-band ratings, 125 toxic-comment decisions, 104 insult, 60 obscenity, 47 threat (all negative) and 44 identity-attack annotations. The result is therefore transfer from definitions to the evaluated Polish benchmarks without benchmark-specific training, not moderation without any related training. Sójka (Bielik Guard) 0.1B v1.1 and 0.5B v1.1 [41], the open Polish guard classifiers, with five sigmoid heads (hate, vulgarity, sexual content, crime, self-harm); the 0.1B model is the size-matched baseline of [40]. Baszta [40], a 124M-parameter HerBERT classifier fine-tuned for the same five categories; its checkpoint is not public, so its numbers are quoted from the paper. All other numbers were measured by us on one H100 80 GB with the same data and the same scoring code.

Table 27: basal-1.0 asked with definitions against Polish guard classifiers: F1 with 95% bootstrap intervals. Default: threshold 0.5. Calibrated: per-head threshold fitted on one third of each set, evaluated on the other two thirds. Best measured value per row in bold. Baszta values are quoted from [40] (Gadzi: calibrated on a 187-item split; other sets: thresholds fitted on the Sójka validation set, CBD and BAN-PL after a contamination filter), and are not paired with ours.

	Set	basal 4.5B	basal 1.5B	Sójka 0.1B	Sójka 0.5B	Baszta [†]
Default	Gadzi micro	0.867 [.845,.889]	0.732 [.699,.765]	0.584 [.545,.620]	0.826 [.798,.851]	—
	Gadzi macro	0.607 [.532,.672]	0.694 [.567,.773]	0.619 [.500,.678]	0.676 [.525,.773]	—
	KLEJ CBD	0.674 [.606,.737]	0.636 [.570,.699]	0.109 [.043,.179]	0.696 [.628,.755]	—
	HateCheck-PL	0.847 [.837,.857]	0.733 [.719,.748]	0.528 [.508,.546]	0.767 [.754,.779]	—
	BAN-PL	0.820 [.815,.825]	0.817 [.812,.823]	0.705 [.697,.712]	0.848 [.843,.852]	—
	PolyGuard-PL	0.417 [.378,.457]	0.289 [.250,.327]	0.291 [.252,.328]	0.438 [.398,.479]	—
Calibrated	Gadzi micro	0.928 [.907,.947]	0.928 [.908,.946]	0.936 [.915,.954]	0.934 [.914,.951]	0.929
	Gadzi macro	0.660 [.505,.773]	0.740 [.582,.837]	0.773 [.575,.856]	0.803 [.616,.877]	0.712
	KLEJ CBD	0.646 [.557,.724]	0.646 [.566,.719]	0.403 [.326,.479]	0.699 [.610,.774]	0.671
	HateCheck-PL	0.880 [.869,.890]	0.850 [.838,.861]	0.824 [.812,.836]	0.849 [.836,.861]	0.644
	BAN-PL	0.843 [.837,.849]	0.820 [.814,.826]	0.762 [.754,.769]	0.846 [.841,.852]	0.667
	PolyGuard-PL	0.706 [.674,.737]	0.645 [.614,.676]	0.632 [.603,.661]	0.652 [.618,.684]	0.562

Benchmarks. The five sets of [40]: Gadzi Język [42] (520 adversarial prompts, multi-label over the five categories), KLEJ CBD [43] (1,000 tweets, cyberbullying, scored on the hate head), HateCheck-PL [44] (3,815 functional tests, hate head), BAN-PL [45] (24,000 moderated posts, scored on “any category”, the maximum over the five heads) and PolyGuardPrompts-PL [46] (1,725 Polish prompts, of which the 1,699 with a harm label are scored; the 26 unlabelled prompts are excluded, any category). Baszta was evaluated on 851 CBD and 23,539 BAN-PL rows after a contamination filter that is not published, so its numbers on these two sets are not strictly paired with ours.

How basal is asked. One request per text. The text is the state; each category is a yes/no question with a one-sentence Polish definition, for example:

Request (one text, five categories)

```
{
  "state": "Tekst do oceny: ...",
  "questions": {
    "hate": {
      "type": "noul",
      "instructions": "Czy tekst zawiera mowę nienawiści lub przemoc słowną: obrażanie, poniżanie, groźby lub atakowanie osób albo grup (np. ze względu na pochodzenie, narodowość, religię, płeć, orientację, niepełnosprawność)?",
    },
    "vulgar": {
      "...",
      "sex": {
        "...",
        "crime": {
          "...",
          "selfharm": {
            "...",
          }
        }
      }
    }
  }
}
```

The answer to each question is a calibrated probability, used exactly like a classifier score.

Protocol. F1 of the positive class; for Gadzi, micro and macro F1 over the five categories. Two operating points, applied identically to every measured system: *default*, a threshold of 0.5 on every head; and *calibrated*, a threshold per head chosen to maximise F1 on a stratified third of each set and applied to the remaining two thirds (the Gadzi calibration split of [40] is 187/333, ours is 173/347; PolyGuard-PL 566/1,133). Confidence intervals are 95% percentile intervals from 1,000 bootstrap resamples of the evaluated items; differences between systems are paired (the same resample for both). As a check of data and scoring, our Sójka-0.1B per-category F1 on Gadzi at the default threshold matches the values reported for this model in [40] to within 0.002 (hate 0.045, vulgarity 0.889, sexual content 0.759, crime 0.589 vs. 0.587, self-harm 0.815, macro 0.619).

Quality. Table 27 and Table 28. At the calibrated operating point basal-1.0-4.5B is significantly better than Sójka-0.1B on KLEJ CBD, HateCheck-PL, BAN-PL and PolyGuard-PL; on Gadzi micro and macro F1 the difference is not established. Against the larger Sójka-0.5B it is significantly better on HateCheck-PL (+0.031) and PolyGuard-PL (+0.053) and significantly worse on Gadzi macro F1 (−0.144); on Gadzi micro F1 (−0.006 [−0.024, +0.010]), BAN-PL (−0.004 [−0.009, +0.002]) and KLEJ CBD (−0.053 [−0.137, +0.035]) the comparison is inconclusive. These intervals test differences from zero, not equivalence: the CBD interval is compatible with a loss of up to 14 points, so we do not claim parity. The published Baszta numbers are lower than ours on HateCheck-PL, BAN-PL and PolyGuard-PL and within 0.03 on Gadzi micro F1 and KLEJ CBD, but they were not produced on the same items or splits and are not tested against ours. At the default threshold, where no labelled data is used at all, the 4.5B model has the best Gadzi micro F1 (0.867) and HateCheck-PL F1 (0.847) of all measured systems: its probabilities are calibrated, so 0.5 is

Table 28: Paired differences in F1 (basal-1.0-4.5B minus the guard), calibrated operating point, with 95% bootstrap intervals. Intervals that exclude zero are in bold; an interval that contains zero means the difference is not established in either direction, not that the systems are equivalent.

Set	4.5B – Sójka 0.1B	4.5B – Sójka 0.5B
Gadzi micro	−0.008 [−0.024, +0.008]	−0.006 [−0.024, +0.010]
Gadzi macro	−0.113 [−0.272, +0.002]	− 0.144 [−0.311, −0.035]
KLEJ CBD	+ 0.243 [+0.145, +0.338]	−0.053 [−0.137, +0.035]
HateCheck-PL	+ 0.056 [+0.048, +0.063]	+ 0.031 [+0.022, +0.039]
BAN-PL	+ 0.081 [+0.074, +0.089]	−0.004 [−0.009, +0.002]
PolyGuard-PL	+ 0.073 [+0.041, +0.105]	+ 0.053 [+0.023, +0.086]

Table 29: Speed on one H100 80 GB, 500 texts, all five categories per text. basal: HTTP server (mode fast), median server latency of sequential requests and throughput with 32 requests in flight. Sójka: PyTorch bf16, median latency of single texts and throughput in batches of 32. Baszta reports no speed.

System	Params	Latency, 1 text	Throughput
basal-1.0-4.5B	4.5B	39.3 ms	40 texts/s
basal-1.0-1.5B	1.5B	17.4 ms	96 texts/s
Sójka 0.1B v1.1	0.1B	4.2 ms	4,994 texts/s
Sójka 0.5B v1.1	0.5B	7.4 ms	3,155 texts/s

already a usable operating point, whereas Sójka-0.1B needs a tuned threshold (CBD 0.109 at 0.5, 0.403 calibrated). The 1.5B model follows the 4.5B model at a small distance; on HateCheck-PL its difference from Sójka-0.5B is +0.001 [−0.008, +0.009].

Where it is weaker. Gadzi macro F1 averages five categories of which three are tiny (4 vulgarity, 18 sexual-content and 31 self-harm positives; the intervals span 0.2–0.3), and it is the one metric where a guard leads significantly. Sójka-0.5B also has higher point estimates on KLEJ CBD and BAN-PL, whose labels follow a platform’s moderation practice (BAN-PL) or annotators’ notion of cyberbullying (CBD) rather than a definition that fits in one sentence. Asking one direct question (“should this text be blocked by moderation?”) instead of taking the maximum of five categories gives 0.848 on BAN-PL at the default threshold (five categories: 0.820), the same point estimate as Sójka-0.5B: the question can be matched to the label policy, a choice a fixed-head classifier does not have.

Speed. Table 29. Here the dedicated encoders win clearly: Sójka-0.1B needs 4.2 ms per text and processes about 5,000 texts per second in batches of 32, while basal-1.0-4.5B needs 39 ms for the five categories and serves 40 texts per second (1.5B: 17 ms and 96 texts per second). The gap is the price of a 4.5B decoder that reads a new taxonomy with every request: for a fixed taxonomy at high volume a trained encoder is the right tool; basal fits where the categories change, are specific to one client or have no training data.

A taxonomy defined per request. A guard classifier answers exactly the questions it was trained on. basal answers the questions in the request, so a moderation policy can add categories, split one into finer ones, ask *who* is targeted or *how severe* the content is, and do so per client, without retraining. We tested this on HateCheck-PL, whose metadata gives gold labels for questions outside the five-category taxonomy, with four new questions in one request (Table 30): the targeted group (a choice among seven protected groups and “none”), a threat of violence (yes/no), a slur (yes/no) and severity (a score from 0 to 3). An example answer:

“Próbuje się dogadać z gejami, ale są po prostu zbyt odpychający.” (hateful; target: gay people)
 target: gay ($p = 0.986$); threat: $p = 0.019$; slur: $p = 0.529$; severity: 1.41 on 0–3 (1: 0.55, 2: 0.40).

The targeted group is identified correctly for 92.5% of the 2,908 labelled texts (1.5B: 89.3%; chance 12.5%) and threats are separated from other content with AUROC 0.913, with no example in the request and no Polish training data for either task (the English fine-tuning data contain 47 threat annotations, all negative, and 44 identity-attack annotations, but no question about which group is targeted). The severity score is on average 2.07 for hateful and 1.16 for non-hateful texts (AUROC 0.79). The slur question shows the other side of the same property: basal answers the question as written, and “an offensive term for a group” is also true of the many derogatory HateCheck-PL items that are not labelled as slurs (mean p 0.94 for dehumanising statements, 0.74 for the slur items), so it separates slurs from non-hateful texts

Table 30: Questions outside the five-category taxonomy, asked from definitions on HateCheck-PL in one request with gold labels from its metadata, and the cost of adding questions (median server latency on an H100, 200 texts, questions added in the order: five categories, “unsafe”, threat, slur, target group, severity, spam, personal data). Measured with engine v1.0, which showed the target-group options by their Polish descriptions only; v1.0.1 would also show the keys (e.g. “gay: osoby homoseksualne”). The yes/no questions of the main comparison are not affected.

New question (type)	basal 4.5B	basal 1.5B
Targeted group (choice of 8), accuracy, $n=2,908$	0.925	0.893
Threat of violence (yes/no), AUROC, 280 positives	0.913	0.833
Slur (yes/no), AUROC vs. non-hateful / overall	0.738 / 0.525	0.651 / 0.463
Severity 0–3 (score), mean hateful / non-hateful	2.07 / 1.16	2.05 / 1.47
Latency, 1 / 2 / 5 / 8 / 12 questions (ms)	10 / 14 / 39 / 42 / 100	5 / 7 / 17 / 19 / 42

(AUROC 0.74) but not from other hate (0.53 overall). A narrow category needs a narrow definition; with a fixed-head classifier the same change needs new labels and a new model.

Cost of more categories. Each question is a separate prompt that contains the text: the engine shares the prefix between the two option orders of one question, not between questions, and runs the questions of a request together in one batch. The text is therefore processed once per question, and latency grows with the number of questions: from 10 ms for one question to 39 ms for the five categories and 100 ms for twelve questions, including an eight-way choice and a four-level score (Table 30). Sharing the text across questions would reduce this cost; it is not implemented in the measured engine.

Decision. Asked with one-sentence definitions and without training on these benchmarks or on Polish moderation data, basal-1.0-4.5B is significantly better than Sójka-0.1B, the baseline guard of [40], on four of five benchmarks; against Sójka-0.5B, a guard trained for exactly these categories, it is significantly better on two metrics, significantly worse on one and inconclusive on three. It also answers questions no fixed guard can (who is targeted, how severe, any new category). It is 5–10× slower per text and has about 100× lower batch throughput; for a fixed taxonomy at very high volume a trained encoder remains the better choice, and a small Polish safety set in training is the obvious next step.

22 Summary of decisions

Table 31: Design decisions and the evidence behind them.

Decision	Evidence
Polish-specific (APT4) tokenizer	Multilingual tokenizers need 17–51% more tokens on Polish prompts and make 15–34% fewer decisions/s at batch 32. About $1.6\times$ more tokens on English (Step 2)
Base family with a shared tokenizer. 4.5B main, 11B teacher	Shared vocabulary allows full-vocabulary distillation (Step 3; decision distillation needs only the option letters). MoE alternative failed on knowledge and determinism (Steps 1, 6)
Compete on coverage at fixed error, not raw ECE	One temperature fixes calibration for every model (Step 1)
No distillation on generic text	–2.0 points knowledge. Calibration gain equals temperature scaling (Step 3)
Decision data with code-computed, grounded or verified labels	4.5B beats Jev on held-out decisions by 7–15 points (Steps 5, 7)
Letter readout + random option order	Flip 0.16→0.03. Consistency loss adds nothing. Pointer head less accurate and cannot compare options (Steps 5, 7)
LoRA on the instruction model, $lr\ 5 \times 10^{-5}$, guarded training	10^{-4} diverged. Full fine-tuning of the base model worse at our data scale (Step 7)
Evidence-sufficiency companion question	0.95–0.96 vs 0.82 on removed evidence (Step 7)
Decompose composite rules into typed primitives	0.48→0.92 (v2) and 1.00 (v4) on filing on time (Steps 7, 9)
English decision data ($\approx 30\%$ in v3, 21% in v4)	Polish unchanged, English 0.756 vs 0.736 for Jev (Step 8)
Distil only from a domain-strong teacher, only where it is right	Weak-in-English teacher hurt English, strong-in-dates teacher helped dates (Step 8)
Twin pairs for rule families	Every pair tests that one changed fact changes the answer. Jev 0.63 vs 0.93 on these families (Step 9)
fp32 evaluation with parity check.	bf16 decisions depended on batch composition (Section 3)
Selection on calibration split	
Frozen version selected on calibration, per-type temperatures fitted on the served (order-averaged) predictor	0.858 test, 58.6% coverage at a frozen 1% target (1.2% observed) (Step 10)
Early exits as an optional per-request setting	$1.22\times$ fewer layers, no measurable test loss (level chosen on calibration). On the optimised path 0.99: 12.2→10.5 ms at unchanged agreement. Target $1.5\times$ missed (Steps 4, 11)
Speed from the serving path first	bf16, compilation, CUDA graphs: 32.9→16.5 ms per two-order decision at 99.5–99.7% agreement with fp32 (Step 12)
Keep two option orders, share the prefix	One order costs 0.4–0.7 points. Packing both orders in one pass: 12.5 ms vs 11.6 ms for one order (Step 12)
Token-budget batching	Padding to the longest prompt wasted up to 2/3 of compute with the rest of the serving path 25→63 decisions/s on one H100 (Step 12)
1.5B distilled as the lite variant.	1.5B: –2.9 points, $2.5\times$ throughput. 36 layers: –8.1. Looped model did not train (Step 11)
No pruning, no looped model	Without compilation slower (27.0 vs 16.5 ms per order). Compiled $1.3\text{--}2.1\times$ lower latency on workstation, consumer and desktop GPUs, no gain on B300, $\approx 3\%$ of decisions changed (Step 12; Benchmarks)
FP8 only with compilation, opt-in	$\approx 10\%$ of decisions changed, –3 points (4.5B). vLLM on DGX Spark $3.2\times$ the batch throughput of FP8 (Benchmarks)
NVFP4 only for batched throughput	v5: +3.9 on the public English benchmark and +27 on new reasoning tasks, but –0.6 / –1.5 on Polish decisions / general (Step 13)
Keep the v4 model as basal-1.0, not v5	

23 Data, licence and limitations

Licence. basal-1.0-4.5B, basal-1.0-1.5B and the inference engine are released under Apache 2.0.

Availability. The models, their FP8 and NVFP4 checkpoints and the engine are public from 1 October 2026 (before that date the repositories are private). The version described here is pinned as revision v1.0.1 in each repository:

- models: <https://huggingface.co/Remek/basal-1.0-4.5B>, <https://huggingface.co/Remek/basal-1.0-1.5B> (with -FP8 and -NVFP4 variants), collected at <https://huggingface.co/collections/Remek/basal-10-6ab8224bf7bd8732d7a6117d>;
- engine, tests and benchmark tools: <https://github.com/rkinas/basal>;
- leaderboard: <https://huggingface.co/spaces/Remek/jev-pl-benchmark> (scores only).

The Polish decision test items are not published, so that they stay out of future training data. The author evaluates other systems on them on request and returns the readouts. The research pipeline (data generators, training, calibration and table scripts, with the manifests and saved predictions behind every number, and both data versions: the original v4 used for training and the corrected v4-fix1 used for evaluation) is not part of the engine repository and is available from the author on request.

Provenance. Every training item records its generator and verifiers (Section 18). In the data of the released models (v4), 51% of the training items are generated by code without any language model, 22% are written and verified by language models (of which the broad decisions, 21%, involve commercial models, used in accordance with the providers’ terms of use) and 27% are converted public data. In v5 the shares are 65%, 16% (15%) and 19%. A variant without the commercially generated or verified items can be produced from the same files by filtering on the provenance field. Statutes and court decisions are not protected by copyright in Poland. The 13,500 converted English items (21.2% of v4) come from an aggregated public corpus that asks users to check and attribute its upstream sources and has no row-level source identifiers. We have not traced these items to their upstream sources, and this provenance is an open limitation of the release. They can be removed by filtering, and the next release will either document their sources or train without them.

Limitations.

- **Same generators in training and test.** Test items come from unseen templates, statutes and domains but from the same generation pipelines as training. Claims about real documents require a human-reviewed benchmark built from real Polish documents with controlled variants. This is the most important remaining step.
- **Easy broad items.** Verifier agreement selects easy broad items (Jev 0.86–0.98 per broad family).
- **Single runs.** Each variant is one run with one seed. Differences smaller than the confidence intervals are not interpreted.
- **Knowledge.** Polish world knowledge of the 4.5B model remains well below Jev (0.58 vs 0.79). Closing it needs a larger model or knowledge supplied in the state.
- **Temporal validity.** Legal rules change. Generators and labels carry a rules version.
- **Adaptive use of the test split.** The v4 test split was consulted during development (Section 3); its results are not those of a single untouched final evaluation. A fresh, case-disjoint holdout generated after freezing, scored once, is planned.
- **Known label error in training.** The released checkpoints learned the old notice-period rule (125 mislabelled items, Section 18). For notice given within one month of three years of employment they answer one month instead of three.
- **Order sensitivity.** Reversal averaging reduces but does not remove sensitivity to option order for three or more options.
- **Quantised checkpoints.** Their calibration is inherited from the bf16 model and not validated.

24 Status and next steps

Done in this research - The frozen model version with its calibration manifest. Trained early exits, shipped as an optional per-request setting. The compression comparison (1.5B lite variant adopted; pruning and the looped model rejected). The optimised serving path (12.5 ms per two-order decision on one H100, 8.8 ms on a B300). FP8 on five GPUs and NVFP4 on the B300 and DGX Spark. A quality comparison with Jev and a quality and speed comparison with eleven open decision systems. The speed of the 1.5B model on six GPUs and of the 4.5B model on five. The v5 reasoning data (evaluated, not adopted). The release of basal-1.0-4.5B and basal-1.0-1.5B with FP8 and NVFP4 checkpoints and the basal inference engine (public on 1 October 2026), with the calibration of the served predictor (v1.0.1). Next: retraining with the corrected notice-period labels. A fresh, case-disjoint holdout generated after freezing and scored once. Tracing the upstream sources of the converted English items or training without them. A data mix that

keeps the v5 reasoning gain without the Polish loss. Quantisation-aware training for a 4-bit model without the quality loss and the human-reviewed Polish benchmark built from real documents.

References

- [1] TypeSafe AI. Introducing System One models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>, 2026. Model documentation: <https://docs.typesafe.ai/model-jaggedness/jev-1.13>. Accessed 2026-09-24.
- [2] heyjunpenn. awesome-jev: a curated list of Jev-like typed-decision projects. <https://github.com/heyjunpenn/awesome-jev>, 2026. Accessed 2026-09-24.
- [3] F. Standhartinger. JevBench: a benchmark for typed-decision models. <https://github.com/fstandhartinger/jevbench>, 2026. Version 1.4. Accessed 2026-09-24.
- [4] Benchmark Heaven. Jevbench v1.4.2 leaderboard. <https://benchmarkheaven.com/jev-models>, 2026.
- [5] Sean Goedecke. Jev means structured output is interesting again. <https://www.seangoedecke.com/jev-means-structured-output-is-interesting-again/>, 2026.
- [6] Jared Palmer. kev: open typed-decision models with packed options and a pointer head. <https://github.com/jaredpalmer/kev>, 2026. Accessed 2026-09-24.
- [7] Ji Xin, Raphael Tang, Jaesun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [8] Tal Schuster et al. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [9] Mostafa Elhoushi et al. LayerSkip: Enabling early exit inference and self-speculative decoding, 2024. arXiv:2404.16710.
- [10] Sangmin Bae et al. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA, 2024. arXiv:2410.20672.
- [11] Guan Wang et al. Hierarchical reasoning model, 2025. arXiv:2506.21734.
- [12] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025.
- [13] allebee. JevK5: an open typed-decision model on Qwen3.5-4B. <https://github.com/allebee/jevK5>, 2026. Version 0.2. Accessed 2026-09-24.
- [14] HopitAI. Hopper: a typed-decision model on Qwen3.5-4B. <https://huggingface.co/HopitAI/hopper>, 2026.
- [15] EldanRing. Winnow-12B: a typed-decision model on Gemma 4 12B. <https://huggingface.co/EldanRing/Winnow-12B>, 2026.
- [16] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.03882.
- [17] NASK-PIB. Reassess polish medical exams (lek subset). <https://huggingface.co/datasets/NASK-PIB/Reassess-Polish-Medical-Exams>, 2025. Accessed 2026-09-24.
- [18] Adam Mickiewicz University (AMU CAI). LLMzSzŁ: a benchmark for Polish based on national exams. <https://huggingface.co/datasets/amu-cai/llmzszl-dataset>, 2025. Accessed 2026-09-24.
- [19] Angelika Romanou et al. INCLUDE: Evaluating multilingual language understanding with regional knowledge, 2024. arXiv:2411.19799.
- [20] Shivalika Singh et al. Global MMLU: Understanding and addressing cultural and linguistic biases in multilingual evaluation, 2024. arXiv:2412.03304.
- [21] Jan Kocoń, Piotr Miłkowski, and Monika Zaśko-Zielińska. Multi-level sentiment analysis of PolEmo 2.0: Extended corpus of multi-domain consumer reviews. In *Proceedings of the Conference on Computational Natural Language Learning (CoNLL)*, 2019.
- [22] David Ifeoluwa Adelani et al. SIB-200: A simple, inclusive, and big evaluation dataset for topic classification in 200+ languages and dialects. In *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. arXiv:2309.07445.

- [23] Alina Wróblewska and Katarzyna Krasnowska-Kieraś. Polish evaluation dataset for compositional distributional semantics models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.
- [24] Lucas Bandarkar et al. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2308.16884.
- [25] SpeakLeash and ACK Cyfronet AGH. Bielik v3: Polish language models (1.5b, 4.5b, 11b). <https://huggingface.co/speakleash>, 2025. Model cards of Bielik-1.5B/4.5B/11B/PL-11B-v3.0-Instruct. Accessed 2026-09-24.
- [26] PLLuM Consortium. PLLuM: Polish large language models. <https://huggingface.co/CYFRAGOVPL>, 2025. Model cards of PLLuM-12B-instruct-2512 and Llama-PLLuM-8B-instruct-2512. Accessed 2026-09-24.
- [27] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [28] Krzysztof Ociepa, Łukasz Flis, Remigiusz Kinas, Krzysztof Wróbel, and Adrian Gwoździej. Bielik v3 Small: Technical report, 2025.
- [29] Konstantin Dobler and Gerard de Melo. FOCUS: Effective embedding initialization for monolingual specialization of multilingual models. In *Proceedings of EMNLP*, 2023. arXiv:2305.14481.
- [30] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. arXiv:1503.02531.
- [31] nostalgebraist. Interpreting GPT: the logit lens. LessWrong, 2020.
- [32] Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens, 2023.
- [33] Mapika. decider: System one-style typed decision models on qwen3.5. <https://github.com/Mapika/decider>, 2026. Apache-2.0; commit 15ab28e; model card <https://huggingface.co/Mapika/decider-4b>.
- [34] A. Biswas. bev-decision-150k. <https://huggingface.co/datasets/avbiswas/bev-decision-150K>, 2026.
- [35] multimodalart. Decision index (edition 0.2). <https://huggingface.co/spaces/multimodalart/jev-decision-index>, 2026.
- [36] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Gloriosi, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers, 2024. arXiv:2403.17887.
- [37] Saurav Muralidharan et al. Compact language models via pruning and knowledge distillation, 2024. arXiv:2407.14679.
- [38] NVIDIA. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, 2025.
- [39] NVIDIA. NVIDIA Model Optimizer (formerly TensorRT Model Optimizer). <https://github.com/NVIDIA/Model-Optimizer>, 2025.
- [40] Adam Górski, Mateusz Jąkalak, and Rafał Jakubowski. Baszta: Data-centric fine-tuning of a Polish multi-label safety classifier, 2026.
- [41] SpeakLeash. Bielik Guard (Sójka): Polish content-safety classifiers. <https://huggingface.co/speakleash/Bielik-Guard-0.1B-v1.1>, 2025.
- [42] Jerzy Surma et al. GadziJezyk: a Polish dataset of toxic prompts. <https://huggingface.co/datasets/JerzyPL/GadziJezyk>, 2024.
- [43] Michał Ptaszynski, Agata Pieciukiewicz, and Paweł Dybała. Results of the PolEval 2019 shared task 6: First dataset and open shared task for automatic cyberbullying detection in Polish Twitter. In *Proceedings of the PolEval 2019 Workshop*, 2019.
- [44] Paul Röttger, Haitham Seelawi, Debora Nozza, Zeerak Talat, and Bertie Vidgen. Multilingual HateCheck: Functional tests for multilingual hate speech detection models. In *Proceedings of the Sixth Workshop on Online Abuse and Harms (WOAH)*, 2022.
- [45] Anna Kołos, Inez Okulska, Kinga Głabińska, Agnieszka Karlińska, Emilia Wiśnios, Paweł Ellerik, and Andrzej Prałat. BAN-PL: A Polish dataset of banned harmful and offensive content from Wykop.pl web service. In *Proceedings of LREC-COLING 2024*, pages 2107–2118, 2024.
- [46] Priyanshu Kumar, Devansh Jain, Akhila Yerukola, Liwei Jiang, Himanshu Beniwal, Thomas Hartvigsen, and Maarten Sap. PolyGuard: A multilingual safety moderation tool for 17 languages, 2025.

A Query examples

Requests sent to the released server (`basal-serve --mode fast`, one H100) and its actual responses, shortened only by rounding, produced with engine and calibration v1.0. The v1.0.1 temperatures (Step 10) change the probabilities slightly but never the chosen answer. v1.0.1 also shows option keys next to their descriptions by default (Section 19), which can change the probabilities of requests with such keys. The requests are in `basal/examples/types_demo.py` of the inference repository.

choice — route a ticket

State: Klient: od wczoraj nie mogę zalogować się do bankowości internetowej, system pokazuje błąd hasła.

Question dept (choice): Do którego działu skierować zgłoszenie?; *criteria:* {"cards": "Reklamacje kart", "online": "Wsparcie bankowości elektronicznej", "loans": "Kredyty"}

⇒ **choice** = online; probabilities: online 0.999, loans 0.000, cards 0.000; confidence 0.999

Server latency: 10.5 ms (1 question, both option orders, H100, mode fast)

noul — a yes/no decision without criteria

State: Umowa zawarta na odległość. Towar odebrano 3 marca 2025 r. Oświadczenie o odstąpieniu od umowy wysłano 20 marca 2025 r. Regulamin: konsument może odstąpić od umowy w terminie 14 dni od objęcia rzeczy w posiadanie; do zachowania terminu wystarczy wysłanie oświadczenia przed jego upływem.

Question in_time (noul): Czy odstąpienie od umowy złożono w terminie?

⇒ **noul** = 0.510 (P(yes)); probabilities: true 0.510, false 0.490; confidence 0.510

Server latency: 10.6 ms (1 question, both option orders, H100, mode fast)

Note: Correct answer: *no* (14 days from 3 March end on 17 March; the statement was sent on 20 March). The model is undecided (0.51) — the confidence is far below the automation thresholds in `CALIBRATION.json`, so this decision would be routed to a person. We show it unedited: a single example is not a measurement, and uncertain answers are what the confidence is for.

score — an ordered scale given as a list

State: Zgłoszenie z oddziału w Gdańsku: od 7:40 nie działa żaden terminal płatniczy, klienci odchodzą od kas, kolejka na 30 osób. Obejście: tylko gotówka.

Question urgency (score): Jak pilne jest to zgłoszenie?; *criteria:* ["niska - można zaplanować", "średnia - w ciągu kilku dni", "wysoka - dziś", "krytyczna - natychmiast"]

⇒ **score** = 2.71 (expected level, 0 = first level); probabilities: 3 0.722, 2 0.269, 0 0.005, 1 0.003; confidence 0.722

Server latency: 12.6 ms (1 question, both option orders, H100, mode fast)

Several typed questions about one JSON state in one request (fan-out)

State: {"ticket": "Dzień dobry, 12.05 zamówiłam czajnik (nr 88412), przyszedł z pękniętą obudową. Proszę o wymianę albo zwrot pieniędzy. Anna Nowak", "customer": {"segment": "VIP", "orders_last_year": 14}}

Question category (choice): Jaki jest rodzaj zgłoszenia?; *criteria:* {"complaint": "Reklamacja towaru", "delivery": "Opóźniona dostawa", "invoice": "Faktura", "other": "Inne"}

⇒ **choice** = complaint; probabilities: complaint 0.997, delivery 0.001, invoice 0.001, other 0.001; confidence 0.997

Question needs_photo (noul): Czy do rozpatrzenia potrzebne jest zdjęcie uszkodzenia?; *criteria:* {"true": "Tak, poprosić o zdjęcie", "false": "Nie, można rozpatrzyć od razu"}

⇒ **noul** = 0.672 (P(yes)); probabilities: true 0.672, false 0.328; confidence 0.672

Question tone (score): Jak zdenerwowana jest klientka?; *criteria:* {"calm": "spokojna", "annoyed": "zirytowana", "angry": "bardzo zdenerwowana"}

⇒ **score** = 1.66 (expected level, 0 = first level); probabilities: angry 0.683, annoyed 0.298, calm 0.019; confidence 0.683

Server latency: 22.4 ms (3 questions, both option orders, H100, mode fast)

Structured instructions and options (JSON objects), English

State: Page: checkout. Fields: [1] button 'Pay now' (enabled); [2] checkbox 'I accept the terms' (unchecked); [3] link 'Back to cart'.

Question next_click (choice): {"goal": "Complete the purchase", "rule": "Required fields must be set before paying."}; *criteria:* {"1": {"element": "[1] Pay now", "role": "button"}, "2": {"element": "[2] I accept the terms", "role": "checkbox", "checked": false}, "3": {"element": "[3] Back to cart", "role": "link"}}

⇒ **choice** = 2; probabilities: 2 0.973, 1 0.015, 3 0.012; confidence 0.973

Server latency: 13.8 ms (1 question, both option orders, H100, mode fast)

A question with 12 options is rejected with HTTP 422 and the message “question 'q': 12 options (supported: 2..10)”.